

湖南大學

HUNAN UNIVERSITY

课 程 名 称： 数据库系统概论

实 验 名 称： 数据库设计与应用开发大作业

专 业 班 级： 计算机科学与技术 **2102** 班

小 组 成 员： 甘晴 llq yjh lpl hz sg

指 导 老 师： 杨超

完 成 时 间： **2023.12**

学校信息管理系统 实验报告

成员分工：

- 数据库设计部分，实验报告：llq,hz
- 软件设计部分：甘晴，lpl,yjh
- 参与讨论：sg

目录

学校信息管理系统 实验报告
一、具体问题
二、数据库设计部分
1. 需求分析
1.1 背景概述
1.2 功能需求
1.3 数据需求
2. 概念结构设计
2.1 实体
2.2 ER图
3. 逻辑设计
3.1 模型选择
3.2 主题划分
3.3 依赖分析
3.4 联系
3.5 用户子模式
4. 物理设计
4.1 具体设计
4.2 关系实体
4.3 存取方式
4.4 存储结构
5.运行测试
三、软件设计部分
1. web技术栈
技术细节
2. 概要设计（功能）
面向群体
基本要求
功能需求
3. 详细设计（代码）
3.1 宿舍管理员模块
3.2 学生模块
3.3 教师模块
3.4 系统管理员模块
3.5 主页模块
3.6 登录模块
3.7 各类信息模块
3.8 导出表模块
3.9 自定义功能模块
4. 界面设计（用户说明）

4.1 主页
4.2 学生端
4.3 教师端
4.4 宿舍管理员端
4.5 系统管理员端
四、总结与思考

一、具体问题

系统分为四个模式：学生模式、宿管模式、教师模式、系统管理员模式。

- 学生模式的功能有：查看个人信息、宿舍报修、建议与反馈、选课、查看所选课程信息、修改密码、退出。
- 宿管模式的功能有：查看个人信息、宿舍管理、住宿管理、出入登记、宿舍报修、建议与反馈、修改密码、退出。
- 教师模式的功能有：查看个人信息、查看授课信息、查看选课学生信息、退出。
- 系统管理员的功能有：宿舍楼管理、宿舍管理、宿管管理、学生管理、住宿管理、寝室分配、教师管理、课程管理。

二、数据库设计部分

1. 需求分析

1.1 背景概述

为了便于更好的管理学生宿舍和学生选课以及人员信息情况，采用软件与数据库相结合的管理模式对学生宿舍报修，学生选课，教师给学生录入成绩等信息进行记录与处理，从而提升管理效率。

- 学生需求：能够进行选课；查看选课情况；申请宿舍报修；对宿舍情况进行建议与反馈；记录本人基本信息，包括住宿情况，选课情况等；
- 教师需求：记录本人基本信息；能够查看自己授课情况；查看选课学生信息；
- 宿管需求：查看个人信息、宿舍管理、学生管理、住宿管理、出入登记、宿舍报修、建议与反馈、修改密码、退出。
- 系统管理员需求：能够对宿舍楼、宿舍、宿管、学生、住宿、寝室、教师、课程等进行管理。

综上，设计一个学校信息管理系统。可以对学生宿舍报修、教师授课、学生选课等进行管理。

1.2 功能需求

学生端

1. 登录
2. 完善个人信息
3. 修改密码
4. 申请宿舍报修
5. 建议与反馈
6. 查看所选课程信息
7. 修改密码
8. 退出

教师端

1. 登录
2. 完善个人信息

3. 修改密码
4. 查看授课信息
5. 查看选课学生信息
6. 退出

宿舍管理员端

1. 登录
2. 完善个人信息
3. 修改密码
4. 宿舍管理
5. 住宿管理
6. 出入登记
7. 宿舍报修
8. 接收建议与反馈
9. 修改密码
10. 退出

系统管理员端

1. 登录
2. 宿舍楼管理
3. 宿舍管理
4. 宿管管理
5. 学生管理
6. 住宿管理
7. 寝室分配
8. 教师管理
9. 课程管理
10. 删库跑路
11. 退出

1.3 数据需求

数据字典：

数据结构描述，反映数据之间的组合关系：

1.数据结构：宿舍楼

- 含义说明：学生信息管理系统的数据结构，定义了一个宿舍楼的有关信息
- 组成：楼号，层数，房间数，类别，居住性别，宿管编号

2.数据结构：宿舍

- 含义说明：学生信息管理系统的数据结构，定义了一个宿舍的有关信息
- 组成：楼号，楼层，宿舍号，总床位数，剩余床位数，单价

3.数据结构：宿舍管理员

- 含义说明：学生信息管理系统的主体数据结构，定义了一个宿舍管理员的有关信息
- 组成：编号，密码，姓名，性别，手机

4.数据结构：学生

- 含义说明：学生信息管理系统的主体数据结构，定义了一个学生的有关信息
- 组成：学号，密码，姓名，性别，出生日期，年级，院系，班级，手机，是否入住

5.数据结构：建议与反馈

- 含义说明：学生信息管理系统的数据结构，定义了一个建议与反馈的有关信息
- 组成：学号，详细信息

6.数据结构：课程

- 含义说明：学生信息管理系统的数据结构，定义了一个课程的有关信息
- 组成：课程号，课程名，学分，授课教师

7.数据结构：教师

- 含义说明：学生信息管理系统的主体数据结构，定义了一个教师的有关信息
- 组成：职工号，教师名，性别，密码

2. 概念结构设计

包含建议与反馈advice、课程course、宿舍管理员admin、宿舍dormitory、宿舍楼floor、学生student、教师teacher等7个实体。每个实体的属性、码如下：

2.1 实体

1.建议与反馈advice：学号，详细信息info。主码：学号stu_num，详细信息info。

字段名	类型	备注
stu_num	char(15)	主码，学号

字段名	类型	备注
info	text	详细信息

2.课程course：课程号cno，课程名cname，学分score，授课教师tno。主码：课程号cno。

字段名	类型	备注
cno	char	主码，课程号
cname	varchar	课程名
score	smallint	学分
tno	char	授课教师

3.宿舍管理员admin：编号admin_num，密码password，姓名name，性别sex，手机phone。主码：编号admin_num。

字段名	类型	备注
admin_num	char	主码，编号
password	char	密码
name	char	姓名
sex	char	性别
phone	char	手机

4.宿舍dormitory：楼号floor_num，楼层号layer，宿舍号room_num，总床位数bed_total，剩余床位数bed_surplus，单价price。主码：楼号floor_num，楼层layer，宿舍号room_num。

字段名	类型	备注
floor_num	char	主码，楼号
layer	int	主码，楼层
room_num	int	主码，宿舍号
bed_total	int	总床位数
bed_surplus	int	剩余床位数
price	int	单价

5.宿舍楼floor：楼号floor_num，层数layer_amount，房间数room_amount，类别category，居住性别sex，宿管编号admin_num。主码：楼号floor_num。

字段名	类型	备注

字段名	类型	备注
floor_num	char	主码，楼号
layer_amount	int	层数
room_amount	int	房间数
category	char	类别
sex	char	居住性别
admin_num	char	宿管编号

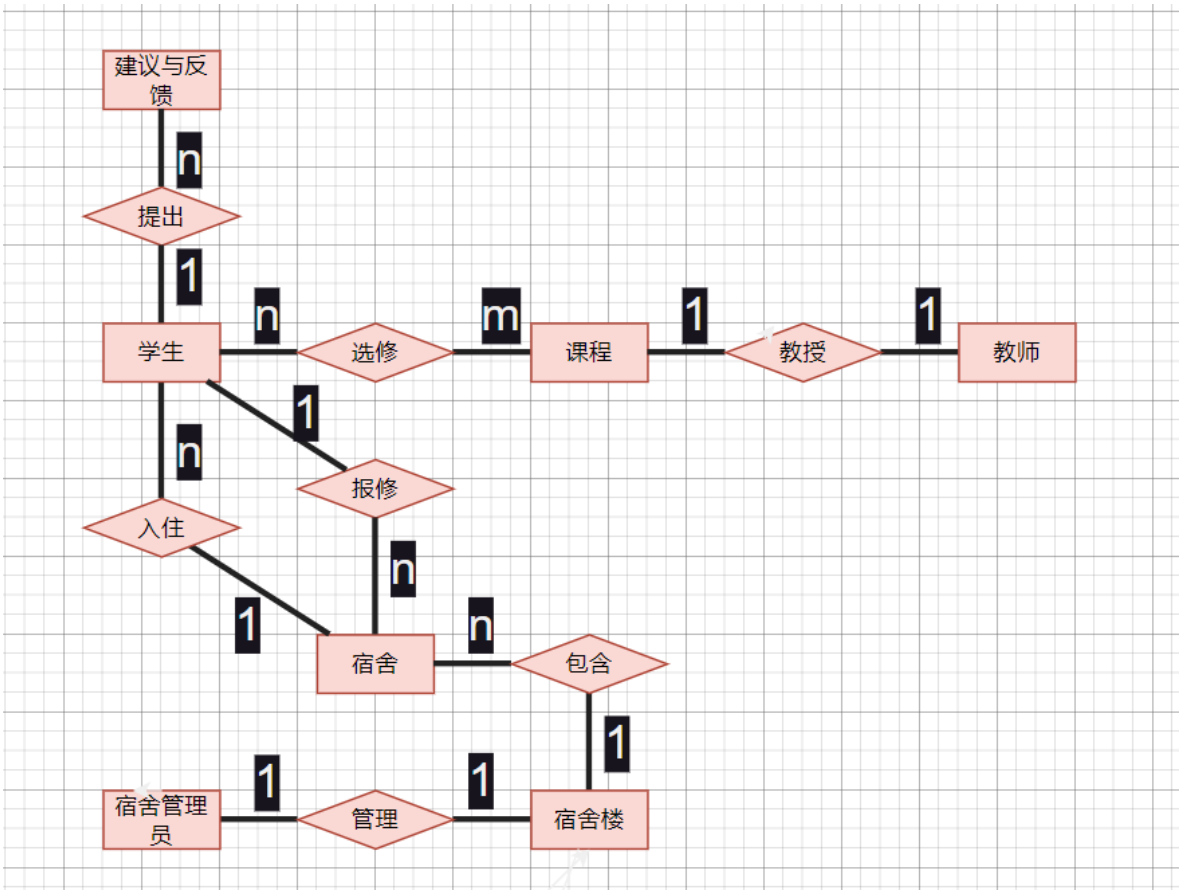
6.学生student：学号stu_num，密码password，姓名name，性别sex，出生日期birth，年级grade，院系faculty，班级class，手机phone，是否入住yes_no。主码：学号stu_num。

字段名	类型	备注
stu_num	char	主码,学号
password	char	密码
name	char	姓名
sex	char	性别
birth	int	出生日期
grade	int	年级
faculty	char	院系
class	char	班级
phone	char	手机
yes_no	char	是否入住

7.教师teacher：职工号tno，教师名tname，性别tsex，密码password。主码：职工号tno。

字段名	类型	备注
tno	char	主码，职工号
tname	varchar	教师名
tsex	char	性别
password	varchar	密码

2.2 ER图



3. 逻辑设计

3.1 模型选择

选用关系数据模型来实现相应的设计。

3.2 主题划分

分成宿舍管理员主题模块和学生教师主题模块，具体如下：

宿舍管理员主题模块

由宿舍管理员、宿舍楼、宿舍、建议与反馈四个实体构成；

1. 宿舍管理员有编号，密码，姓名，性别，手机五个属性；
2. 宿舍楼有楼号，层数，房间数，类别，居住性别，宿管编号六个属性；
3. 宿舍有楼号，楼层，宿舍号，总床位数，剩余床位数，单价六个属性；
4. 建议与反馈有学号，详细信息两个属性；

学生教师主题模块

由学生、课程、教师三个实体构成

1. 学生有学号，密码，姓名，性别，出生日期，年级，院系，班级，手机，是否入住十个属性；
2. 课程有课程号，课程名，学分，授课教师四个属性；
3. 教师有职工号，教师名，性别，密码四个属性；

3.3 依赖分析

1.宿舍管理员（编号，密码，姓名，性别，手机）

由于每个宿舍管理员只有一个候选码编号，所以不存在非主属性对码的部分函数依赖；另外，由于除了候选码以外的所有非主属性每一个都是只有通过候选码推出，因此，也不存在传递依赖，而且因为候选码编号是唯一的决定因素，所以该关系模式也满足BCNF。

2.宿舍楼（楼号，层数，房间数，类别，居住性别，宿管）

由于每个课程只有一个候选码楼号，所以不存在非主属性对码的部分函数依赖；另外，由于除了候选码以外的所有非主属性每一个都是只有通过候选码推出，因此，也不存在传递依赖，而且因为候选码编号是唯一的决定因素，所以该关系模式也满足BCNF。

3.宿舍（楼号，楼层，宿舍号，总床位数，剩余床位数，单价）

由于每个宿舍有三个候选码楼号，楼层，宿舍号。所有的非主属性对每一个码都是完全函数依赖；但是不是所有的主属性对每一个不包含它的码也是完全函数依赖。所以该关系模式不满足BCNF，但是是3NF。

4.建议与反馈（学号，详细信息）

由于建议与反馈的候选码为全码，由于不是所有的主属性对每一个不包含它的码也是完全函数依赖。所以该关系模式不满足BCNF，但是是3NF。

5.学生（学号，密码，姓名，性别，出生日期，年级，院系，班级，手机，是否入住）

由于每个学生只有一个候选码编号，所以不存在非主属性对码的部分函数依赖；但是，院系和班级之间存在传递依赖关系，所以该关系模式满足1NF。

6.课程（课程号，课程名，学分，授课教师）

由于每个课程只有一个候选码课程号，所以不存在非主属性对码的部分函数依赖；另外，由于除了候选码以外的所有非主属性每一个都是只有通过候选码推出，因此，也不存在传递依赖，而且因为候选码编号是唯一的决定因素，所以该关系模式也满足BCNF。

7.教师（职工号，教师名，性别，密码）

由于每个教师只有一个候选码职工号，所以不存在非主属性对码的部分函数依赖；另外，由于除了候选码以外的所有非主属性每一个都是只有通过候选码推出，因此，也不存在传递依赖，而且因为候选码职工号是唯一的决定因素，所以该关系模式也满足BCNF。

8.选课（学号，课程号，成绩）

由于选课的候选码有两个学号和课程号，由于不存在非主属性对码的部分函数依赖；另外，由于除了候选码以外的所有非主属性每一个都是只有通过候选码推出，因此，也不存在传递依赖；但是不符合所有的主属性对每一个不包含它的码也是完全函数依赖，所以该关系模式不满足BCNF，但是是3NF。

9.出入登记（学号，楼号，类别（出/入），时间）

由于出入登记的候选码为全码，由于不是所有的主属性对每一个不包含它的码也是完全函数依赖。所以该关系模式不满足BCNF，但是是3NF。

10.宿舍报修（学号，楼号，楼层，宿舍号，报修详细信息，是否处理（是/否））

由于宿舍报修的候选码为全码，由于不是所有的主属性对每一个不包含它的码也是完全函数依赖。所以该关系模式不满足BCNF，但是是3NF。

11.入住信息（学号，楼号，楼层，宿舍号，入住时间）

由于入住信息的候选码为全码，由于不是所有的主属性对每一个不包含它的码也是完全函数依赖。所以该关系模式不满足BCNF，但是是3NF。

3.4 联系

实体间的联系如下：

一对一：

- 宿舍楼-宿舍管理员
- 教师-课程

一对多：

- 宿舍楼-宿舍
- 宿舍-学生
- 学生-建议与反馈
- 学生-宿舍楼
- 学生-宿舍

多对多：

- 学生-课程

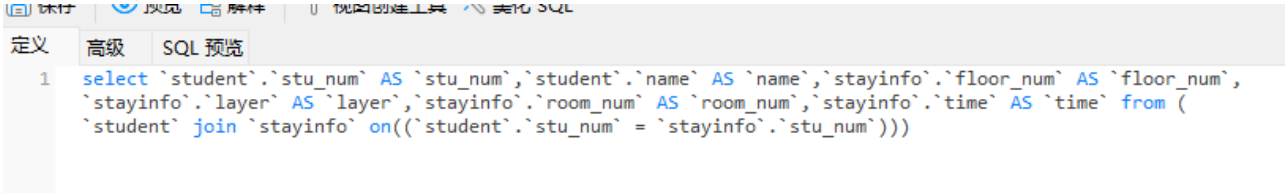
3.5 用户子模式

创建用户子模式，因为宿舍管理员需要频繁查询学生的入住和报修信息等等，为简化查询，并同时保证学生密码及相关隐私信息的安全性。

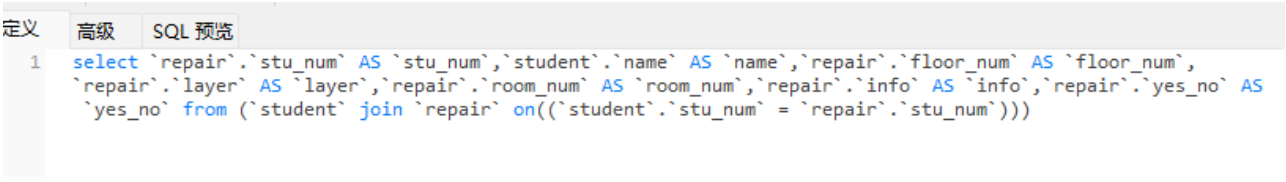
我们可以通过创建视图。

基于我们设计的基本表，设计了6个视图。

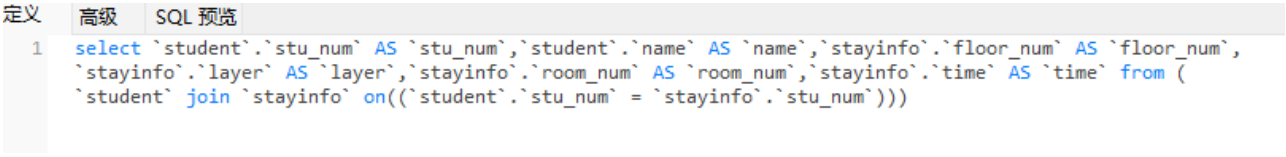
（1）含学生姓名的住宿信息（学号，姓名，楼号，楼层，宿舍号，入住时间）



（2）含学生姓名的出入信息（学号，姓名，楼号，类别，时间）



（3）含学生姓名的报修信息（学号，姓名，楼号，楼层，宿舍号，报修详细，是否处理）



（4）含学生姓名的建议与反馈（学号，姓名，详细信息）

定义 高级 SQL 预览

```
select `advice`.`stu_num` AS `stu_num`,`student`.`name` AS `name`,`advice`.`info` AS `info` from (`student`
join `advice` on((`student`.`stu_num` = `advice`.`stu_num`)))
```

（5）含楼号的住宿学生信息（学号，密码，姓名，性别，出生年份，年级，院系，班级，手机，是否入住，楼号）

```
select `student`.`stu_num` AS `stu_num`,`student`.`password` AS `password`,`student`.`name` AS `name`,
`student`.`sex` AS `sex`,`student`.`birth` AS `birth`,`student`.`grade` AS `grade`,`student`.`faculty` AS
`faculty`,`student`.`class` AS `class`,`student`.`phone` AS `phone`,`student`.`yes_no` AS `yes_no`,
`stayinfo`.`floor_num` AS `floor_num` from (`student` join `stayinfo` on((`student`.`stu_num` = `stayinfo`.
`stu_num`)))
```

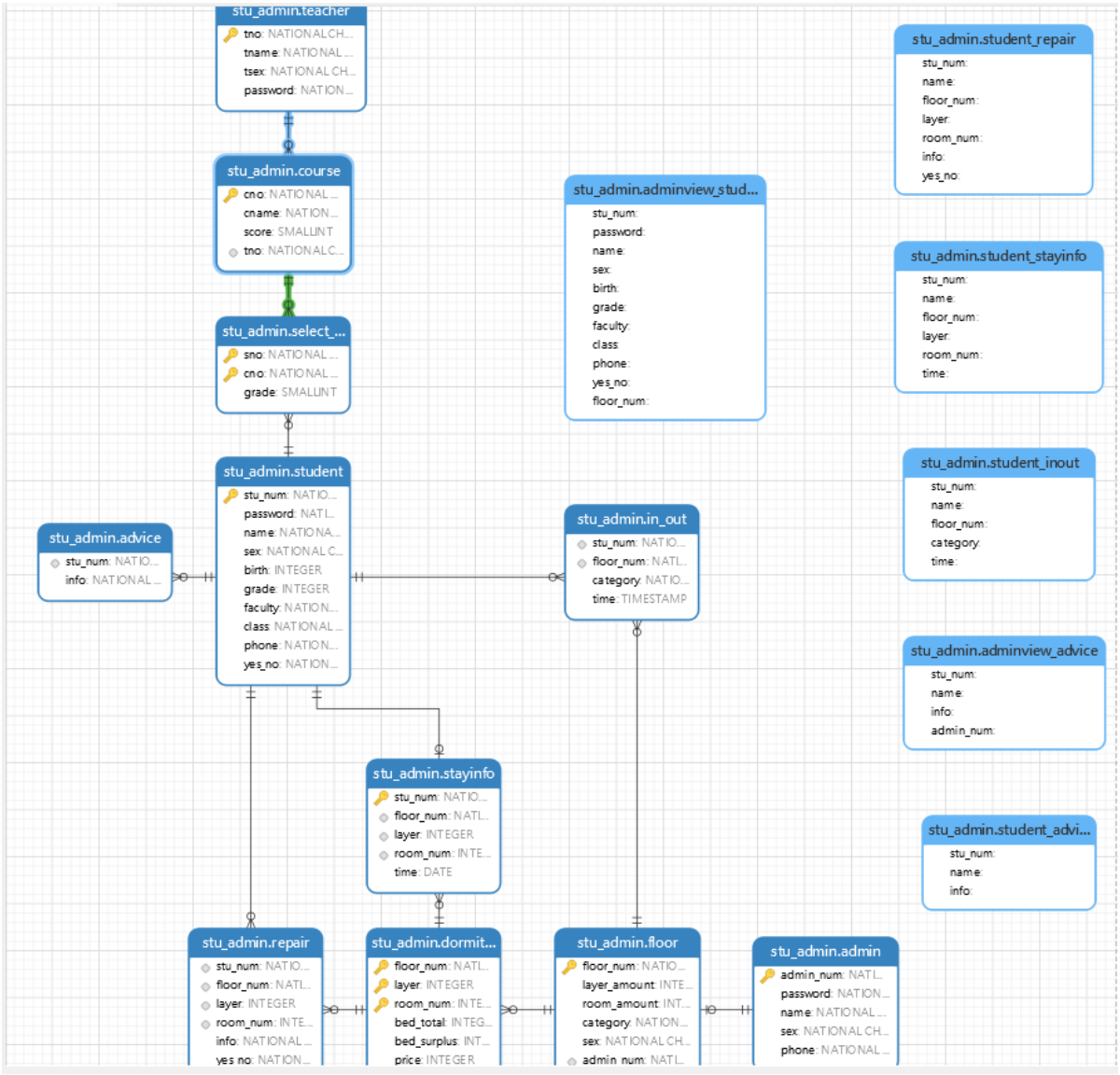
（6）含学生姓名和楼号的住宿学生的建议与反馈（学号，姓名，详细信息，楼号）

```
select `advice`.`stu_num` AS `stu_num`,`student`.`name` AS `name`,`advice`.`info` AS `info`,`floor`.
`admin_num` AS `admin_num` from (((`advice` join `student` on((`advice`.`stu_num` = `student`.`stu_num`)))
join `stayinfo` on((`advice`.`stu_num` = `stayinfo`.`stu_num`))) join `floor` on((`stayinfo`.`floor_num` =
`floor`.`floor_num`)))
```

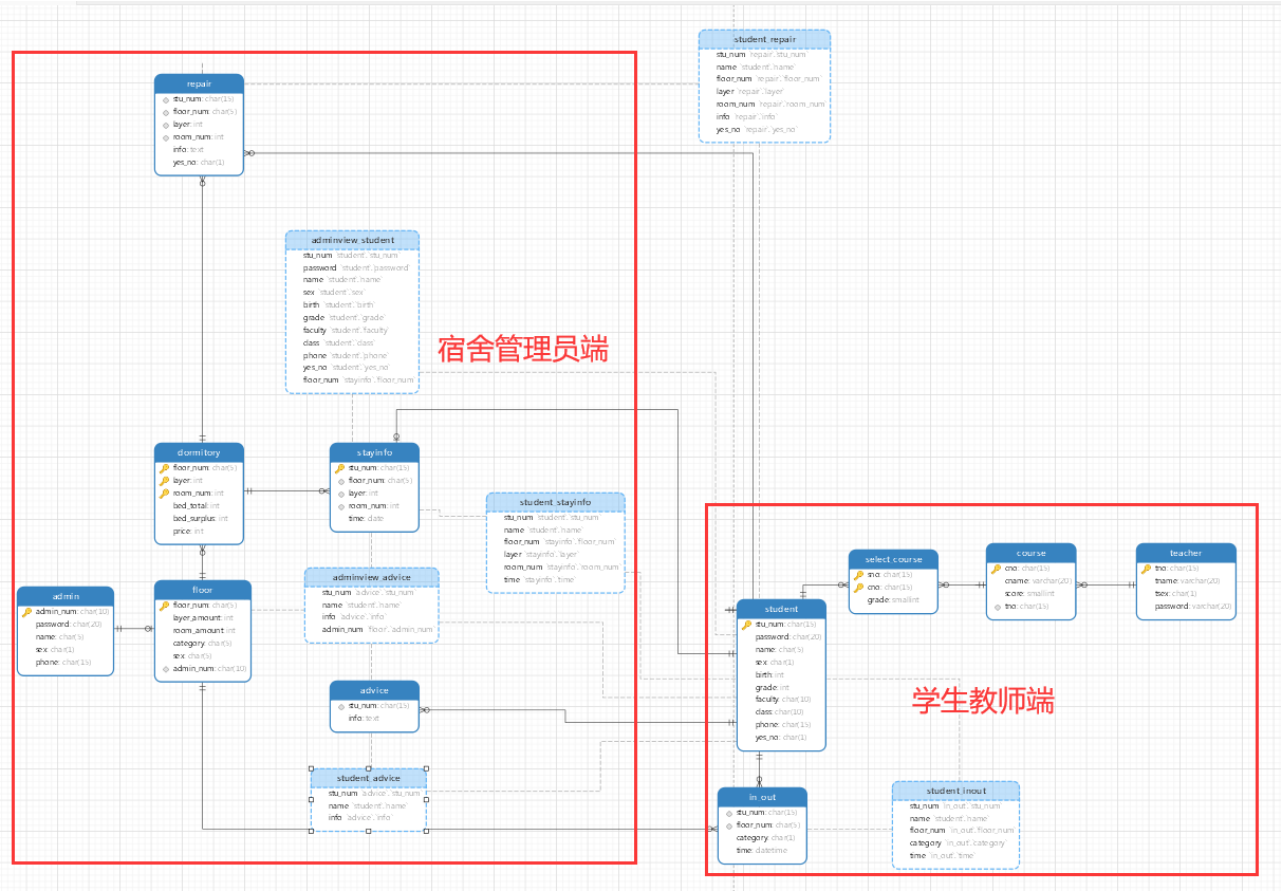
4. 物理设计

4.1 具体设计

根据优化和调整，最后的逻辑模型如下：


stu_admin.teacher
tno: NATIONAL CH...
tname: NATIONAL ...
tsex: NATIONAL CH...
password: NATION...
stu_admin.course
cno: NATIONAL ...
cname: NATION...
score: SMALLINT
tno: NATIONAL C...
stu_admin.select_...
sno: NATIONAL ...
cno: NATIONAL ...
grade: SMALLINT
stu_admin.student
stu_num: NATIO...
password: NATI...
name: NATIONA...
sex: NATIONAL C...
birth: INTEGER
grade: INTEGER
faculty: NATION...
class: NATIONAL ...
phone: NATION...
yes_no: NATION...
stu_admin.advice
stu_num: NATIO...
info: NATIONAL ...
stu_admin.in_out
stu_num: NATIO...
floor_num: NATI...
category: NATIO...
time: TIMESTAMP
stu_admin.stayinfo
stu_num: NATIO...
floor_num: NATI...
layer: INTEGER
room_num: INTE...
time: DATE
stu_admin.repair
stu_num: NATIO...
floor_num: NATI...
layer: INTEGER
room_num: INTE...
info: NATIONAL ...
yes_no: NATION...
stu_admin.dormit...
floor_num: NATI...
layer: INTEGER
room_num: INTE...
bed_total: INTEG...
bed_surplus: INT...
price: INTEGER
stu_admin.floor
floor_num: NATIO...
layer_amount: INTE...
room_amount: INT...
category: NATION...
sex: NATIONAL CH...
admin_num: NATI...
stu_admin.admin
admin_num: NATI...
password: NATION...
name: NATIONAL ...
sex: NATIONAL CH...
phone: NATIONAL ...
stu_admin.adminview_stud...
stu_num:
password:
name:
sex:
birth:
grade:
faculty:
class:
phone:
yes_no:
floor_num:
stu_admin.student_repair
stu_num:
name:
floor_num:
layer:
room_num:
info:
yes_no:
stu_admin.student_stayinfo
stu_num:
name:
floor_num:
layer:
room_num:
time:
stu_admin.student_inout
stu_num:
name:
floor_num:
category:
time:
stu_admin.adminview_advice
stu_num:
name:
info:
admin_num:
stu_admin.student_advi...
stu_num:
name:
info:

最后的物理模型如下：



4.2 关系实体

一个关系对应一张表，根据最后的物理模型，我们需要十一表来实现关系数据模型。

1.宿舍管理员实体：

各属性类型长度定义

字段 索引 外键 检查 触发器 选项 注释 SQL 预览

名	类型	长度	小数点	不是 null	虚拟	键	注释
admin_num	char	10		☑	☐	1	编号
password	char	20		☑	☐		密码
name	char	5		☑	☐		姓名
sex	char	1		☑	☐		性别
phone	char	15		☑	☐		手机

主码为：宿舍管理员编号admin_num；

各属性值的约束

检查
(`sex` = _utf8mb3'男') or (`sex` = _utf8mb3'女')

其中宿舍管理员密码password默认值为'123456'，宿舍管理员性别sex默认值为'男'

2.建议与反馈实体：

各属性类型长度定义



名	类型	长度	小数点	不是 null	虚拟	键	注释
stu_num	char	15		☑	☐		学号
info	text			☑	☐		详细信息

主码为：学号stu_num和详细信息info；

外码为：学号stu_num；

3.教师实体：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
tno	char	15		☑	☐	🔑 1	职工号
tname	varchar	20		☑	☐		教师名
tsex	char	1		☑	☐		性别
password	varchar	20		☐	☐		密码

主码为：工号tno；

各属性值的约束

名	检查	不强制实施
tsex_range	(`tsex` = _utf8mb3'男') or (`tsex` = _utf8mb3'女')	☐

其中教师性别tsex默认值为'男'，密码password默认值为'123456'

4.课程实体：

各属性类型长度定义

字段 | 索引 | 外键 | 检查 | 触发器 | 选项 | 注释 | SQL 预览

名	类型	长度	小数点	不是 null	虚拟	键	注释
cno	char	15		☑	☐	🔑 1	课程号
cname	varchar	20		☑	☐		课程名
score	smallint			☐	☐		学分
tno	char	15		☐	☐		授课教师

主码为：课程号cno；

外码为：教师工号tno;

各属性值的约束

名	检查
score_range	(`score` >= 0) and (`score` <= 100)

其中学分score默认值为NULL

5.宿舍楼实体：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
floor_num	char	5		☑	☐	🔑 1	楼号
layer_amount	int			☑	☐		层数
room_amount	int			☑	☐		房间数
category	char	5		☑	☐		类别
sex	char	5		☑	☐		居住性别
admin_num	char	10		☑	☐		宿管编号

主码为：宿舍楼号floor_num；

外码为：宿舍管理员编号admin_num;

各属性值的约束

名	检查	不强制实施
layer_amount_range	`layer_amount` > 0	☐
room_amount_range	`room_amount` > 0	☐
sex_ran	(`sex` = _utf8mb3'男') or (`sex` = _utf8mb3'女') or (`sex` = _utf8mb3	☐

其中居住性别sex默认值为'男'

6.宿舍实体：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
floor_num	char	5		☑	☐	1	楼号
layer	int			☑	☐	2	楼层
room_num	int			☑	☐	3	宿舍号
bed_total	int			☑	☐		总床位数
bed_surplus	int			☑	☐		剩余床位数
price	int			☑	☐		单价

主码为：宿舍楼号floor_num，楼层号layer，宿舍号room_num；

外码为：宿舍楼号floor_num;

各属性值的约束

名	检查	不强制实施
bed_total_range	`bed_total` > 0	☐
layer_range	`layer` > 0	☐
price_range	`price` > 0	☐
room_num_range	`room_num` > 0	☐

其中总床位数bed_total默认值为4，剩余床位数bed_surplus默认值为4

7.学生实体：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
stu_num	char	15		☑	☐	🔑 1	学号
password	char	20		☑	☐		密码
name	char	5		☑	☐		姓名
sex	char	1		☑	☐		性别
birth	int			☑	☐		出生日期（用于计算年龄）
grade	int			☑	☐		年级
faculty	char	10		☑	☐		院系
class	char	10		☑	☐		班级
phone	char	15		☑	☐		手机
yes_no	char	1		☑	☐		是否入住（是/否）

主码为：学号stu_num；

各属性值的约束

名	检查	不强制实施
stu_sex_range	(`sex` = _utf8mb3'男') or (`sex` = _utf8mb3'女')	☐
birth_range	`birth` > 0	☐
grade_range	`grade` > 0	☐

其中密码password默认值为'123456'，学生性别sex默认值为'男'

8.学生-课程（选课）关系：

各属性类型长度定义

字段 | 索引 | 外键 | 检查 | 触发器 | 选项 | 注释 | SQL 预览

名	类型	长度	小数点	不是 null	虚拟	键	注释
sno	char	15		☑	☐	🔑 1	学号
cno	char	15		☑	☐	🔑 2	课程号
mark	smallint			☐	☐		成绩

其中成绩mark默认值为NULL

主码为：学号sno，课程号cno；

外码为：学号sno，课程号cno；

9.学生-宿舍（出入信息）关系：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
stu_num	char	15		☑	☐		学号
floor_num	char	5		☑	☐		楼号
category	char	1		☑	☐		类别（出/入）
time	datetime			☑	☐		时间

主码为：宿舍楼号floor_num，学号stu_num，类别（出/入）category，时间time；

外码为：宿舍楼号floor_num，学号stu_num；

各属性值的约束

名	检查	不强制实施
category_range	(`category` = _utf8mb3'出') or (`category` = _utf8mb3'入')	☐

10.学生-宿舍（入住信息）关系：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
stu_num	char	15		☑	☐	🔑 1	学号
floor_num	char	5		☑	☐		楼号
layer	int			☑	☐		楼层
room_num	int			☑	☐		宿舍号
time	date			☑	☐		入住时间

主码为：学号stu_num；

外码为：宿舍楼号floor_num，楼层号layer，宿舍号room_num，学号stu_num；

11.学生-宿舍（宿舍报修）关系：

各属性类型长度定义

名	类型	长度	小数点	不是 null	虚拟	键	注释
stu_num	char	15		☑	☐		学号
floor_num	char	5		☑	☐		楼号
layer	int			☑	☐		楼层
room_num	int			☑	☐		宿舍号
info	text			☑	☐		报修详细信息
yes_no	char	1		☑	☐		是否处理（是/否）

主码：全码

外码：楼层号layer，宿舍号room_num，宿舍楼号floor_num，学号stu_num

名	检查	不强制实施
yes_no_range	(`yes_no` = _utf8mb3'是') or (`yes_no` = _utf8mb3'否')	☐

4.3 存取方式

对于多用户共享系统，对同一个关系要建立多条存取路径才能满足多用户的多种应用要求。数据库管理系统一般提供多种存取方法。常用的存取方法分为三类：

1. 索引方法，主要为 B+树索引方法；
2. 聚簇方法；
3. Hash 方法。

在我们的设计中主要采用 B+树索引，利用 using btree 关键字来实现。

代码示例：

```sql
DROP TABLE IF EXISTS `dormitory`;
CREATE TABLE `dormitory`  (
  `floor_num` char(5) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT '楼号',
  `layer` int NOT NULL COMMENT '楼层',
  `room_num` int NOT NULL COMMENT '宿舍号',
  `bed_total` int NOT NULL COMMENT '总床位数',
  `bed_surplus` int NOT NULL COMMENT '剩余床位数',
  `price` int NOT NULL COMMENT '单价',
  PRIMARY KEY (`floor_num`, `layer`, `room_num`) USING BTREE,
  INDEX `floor_num`(`floor_num` ASC) USING BTREE,
  INDEX `layer`(`layer` ASC) USING BTREE,
  INDEX `room_num`(`room_num` ASC) USING BTREE,
  CONSTRAINT `dormitory_FK_floornum` FOREIGN KEY (`floor_num`) REFERENCES `floor` (`floor_num`) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB CHARACTER SET = utf8mb3 COLLATE = utf8mb3_general_ci ROW_FORMAT = DYNAMIC;

-- ----------------------------
-- Records of dormitory
-- ----------------------------
INSERT INTO `dormitory` VALUES ('C16', 2, 263, 4, 2, 150);
INSERT INTO `dormitory` VALUES ('C16', 2, 264, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C16', 2, 265, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C16', 3, 321, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C16', 3, 322, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C17', 1, 101, 4, 3, 150);
INSERT INTO `dormitory` VALUES ('C17', 1, 102, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C17', 1, 103, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C17', 2, 201, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C17', 3, 301, 4, 4, 150);
INSERT INTO `dormitory` VALUES ('C20', 5, 501, 2, 2, 450);
INSERT INTO `dormitory` VALUES ('C20', 5, 520, 2, 0, 450);
```

4.4 存储结构

在数据库中，数据表的存储结构是由存储引擎决定的。存储引擎可以是 MySQL 的 InnoDB 引擎，也可以是其他的存储引擎。

在我们的存储结构中，使用了 InnoDB 存储引擎，它是一种高效的存储引擎，它支持事务，并且

支持多线程。通过 ENGINE = InnoDB sql 语句来实现。

在我们的存储格式中，使用了 Row_FORMAT 行格式，并采用动态的组织格式，即 Dynamic。通过ROW_FORMAT = Dynamic sql 语句来实现。

对于排序的规则使用的是 utf8mb3_general_ci 字符集，通过 CHARACTER SET = utf8mb3 COLLATE= utf8mb3_general_ci sql 语句来实现。凡是涉及到字符类型比较或排序的地方，都会和 COLLATE有关。

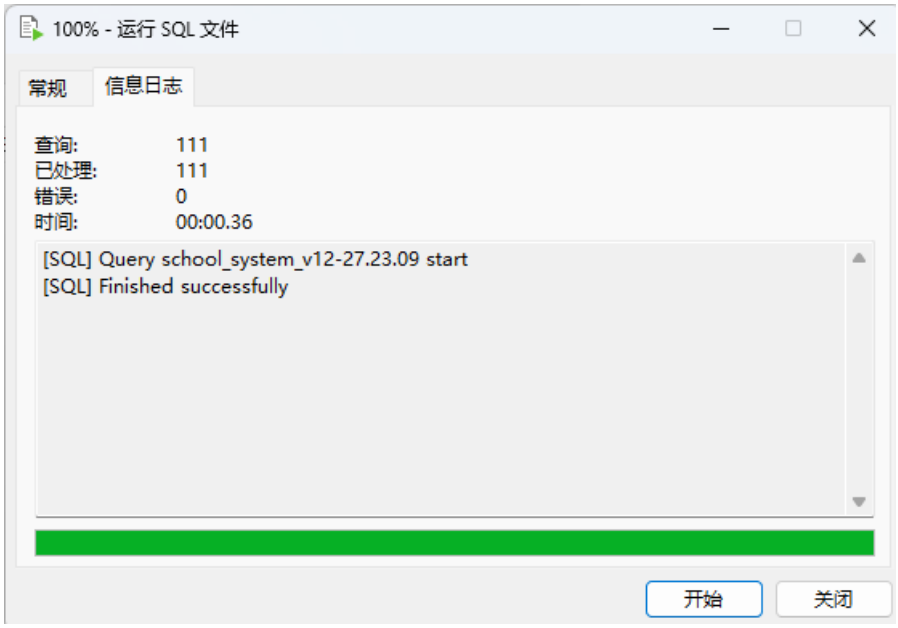
代码示例：

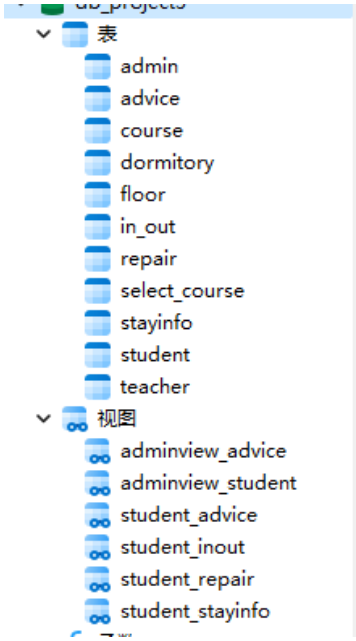
```sql
DROP TABLE IF EXISTS `floor`;
CREATE TABLE `floor`  (
  `floor_num` char(5) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT '楼号',
  `layer_amount` int NOT NULL COMMENT '层数',
  `room_amount` int NOT NULL COMMENT '房间数',
  `category` char(5) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT '类别',
  `sex` char(5) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT '居住性别',
  `admin_num` char(10) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT '宿管编号',
  PRIMARY KEY (`floor_num`) USING BTREE,
  UNIQUE INDEX `admin_num`(`admin_num` ASC) USING BTREE,
  CONSTRAINT `fk_floor_admin` FOREIGN KEY (`admin_num`) REFERENCES `admin` (`admin_num`) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB CHARACTER SET = utf8mb3 COLLATE = utf8mb3_general_ci ROW_FORMAT = DYNAMIC;

-- ----------------------------
-- Records of floor
```

5.运行测试

执行逻辑模型生成的SQL文件，运行成功，结果如下：





1.宿舍管理员信息



admin_num	password	name	sex	phone
1001	010202	刘小贰	男	4286
1002	010206	梅老陆	男	42866
1003	010214	黄拾肆	女	43166
1004	010218	龙拾捌	男	431666
1005	010229	袁贰玖	男	428666
1006	010231	森叁壹	男	42886

我们插入六条记录，分别代表六名管理员，其中宿舍管理员编号为主码，须唯一区分宿舍管理员。

2.管理员--宿舍楼

floor_num	layer_amount	room_amount	category	sex	admin_num
TM11	8	320	普通	男	1004
TM25	7	140	豪华	混合	1001
TM316	7	280	普通	男	1003
TM39	6	240	简朴	女	1002
TM47	7	210	豪华	女	1005
TM66	5	100	巨豪华	男	1006

我们插入六条宿舍楼记录，管理员编号从1001至1006，满足管理员--宿舍楼的一对一关系。

3.宿舍楼--宿舍

开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表

floor_num	layer	room_num	bed_total	bed_surplus	price
TM25	1	428	4	1	300
TM25	2	429	4	2	310
TM25	3	430	4	3	320
TM25	4	431	4	4	330
TM25	5	432	4	0	340
TM316	3	104	4	0	110
TM316	4	103	4	4	120
TM316	5	102	4	3	130
TM316	6	101	4	2	140
TM316	7	433	4	1	150
TM39	1	105	2	1	50
TM39	2	106	2	0	50

插入多条记录，为宿舍楼分配多个宿舍，其中宿舍楼名以TM开头，第三列为宿舍名，满足宿舍楼--宿舍的一对多关系，且该关系符合现实情况。并且现实中宿舍还存在剩余床位的情况，代表没有学生入住该床位。

4.学生信息

stu_num	password	name	sex	birth	grade	faculty	class	phone	yes_no
202008010206	0206	阿梅	男	2001	2020	信息科学与工程学院	计科2002	15651516155	否
202008010214	0214	阿政	女	2002	2020	信息科学与工程学院	计科2001	13692875100	否
202008010218	0218	小强	男	2002	2020	信息科学与工程学院	计科2102	13692873211	否
202108010202	0202	小刘	男	2002	2021	信息科学与工程学院	计科2102	13692871011	是
202108010206	0206	小梅	男	2002	2021	信息科学与工程学院	计科2102	13692871022	是
202108010214	0214	小黄	女	2003	2021	信息科学与工程学院	计科2102	13692871033	是
202108010218	0218	小龙	女	2003	2021	信息科学与工程学院	计科2102	13692875111	是
202108010229	0229	小袁	男	2002	2021	信息科学与工程学院	计科2102	13692873222	是
202108010231	0231	小森	男	2002	2021	信息科学与工程学院	计科2102	13692873033	否

我们插入多条记录，分别代表多名学生，其中学号为主码，须唯一区分学生。

5.学生--入住--宿舍楼（宿舍）

stu_num	floor_num	layer	room_num	time
202008010218	TM39	2	106	2020-09-01
202108010202	TM25	1	428	2021-09-01
202108010206	TM25	1	428	2021-09-01
202108010214	TM316	7	433	2021-09-01
202108010229	TM39	2	106	2021-09-01

插入了五条学生入住信息的记录，其中第二、三条记录的宿舍号均为428，现实中每间寝室可以容纳多个学生，满足寝室与学生是一对多关系。

6.学生出入信息

stu_num	floor_num	category	time
202108010202	TM25	出	2021-09-30 15:30:21
202108010202	TM25	入	2021-10-07 10:45:01
202108010214	TM316	出	2021-11-13 09:25:00

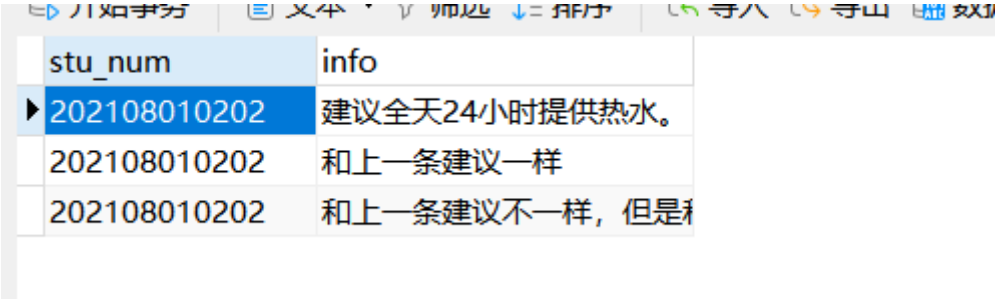
插入了三条记录，代表学生出入宿舍楼的情况，其中包含时间记录，包含具体的某时某刻，满足宿舍楼与学生是一对多关系。每名学生每出或入一次宿舍楼，就记录一次。

7.宿舍报修情况

stu_num	floor_num	layer	room_num	info	yes_no
202108010202	TM25	1	428	宿舍门锁损坏。	否
202108010202	TM25	1	428	厕所堵塞。	否
202108010214	TM316	7	433	空调损坏。	是
202108010206	TM25	1	428	窗户破损。	否
202008010218	TM39	2	106	水龙头损坏。	否

插入了五条宿舍报修记录，报修时需写明原因，并且记录该报修是否处理。

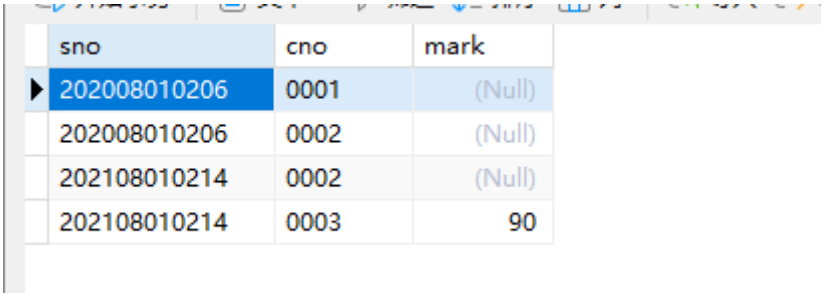
8.学生--建议与反馈



stu_num	info
202108010202	建议全天24小时提供热水。
202108010202	和上一条建议一样
202108010202	和上一条建议不一样，但是

插入了三条记录，包含三条建议与反馈信息，每名学生可以多条提出建议与反馈，满足学生与反馈与信息是一对多关系。

9.学生选课信息



sno	cno	mark
202008010206	0001	(Null)
202008010206	0002	(Null)
202108010214	0002	(Null)
202108010214	0003	90

插入了四条记录，代表四条学生选课信息，可以看到一名学生可以选修多门课程，每门课程也可以被多名学生选修，满足学生与课程的多对多关系，同时教师可以对选修所教授课程的学生录入成绩，成绩初始值为null。

三、软件设计部分

1. web技术栈

- Java
- Java.Swing
- JDBC
- MySQL

技术细节

1. **Java**语言： 代码是使用Java编写的，这是一种面向对象的编程语言，特别适用于跨平台应用程序的开发。
2. **Swing**库： 通过`javax.swing`包，使用了Swing库来构建GUI界面，包括`JFrame`、`JPanel`、`JLabel`、`JButton`、`JTextField`等组件。
3. 数据库连接： 使用了JDBC（Java Database Connectivity）来连接MySQL数据库。这可以看出通过`java.sql`包中的`Connection`、`DriverManager`等类来实现数据库连接。
4. 日志记录： 使用了Java的`java.util.logging`包进行日志记录。在代码中使用了`Logger`和`LoggerUtil`。
5. **MVC**模式： 代码中尽可能遵循了MVC（Model-View-Controller）模式，通过划分代码为不同的功能层次，提高了代码的可维护性和可扩展性。
6. 图形界面定制： 使用了Swing库提供的方法对界面进行了定制，包括字体、颜色、布局等方面。
7. 事件处理： 使用了Java的事件处理机制，例如`ActionListener`和`MouseListener`来处理按钮点击等用户交互事件。
8. 数据库： 应用程序连接到了MySQL数据库，这涉及到使用MySQL JDBC驱动器和SQL语句执行数据库操作。
9. 外观切换： 实现了更换应用程序外观的功能，通过调用Swing的`updateComponentTreeUI`方法刷新界面外观。

2. 概要设计（功能）

面向群体

- 在校宿舍管理员、学生和教师（有实现超级管理员）

基本要求

1. 用户登录
2. 查看个人信息
3. 查看

功能需求

1.学生

- 登录
- 查看个人信息
- 选课
- 查看我的课程
- 进行宿舍报修
- 提交宿舍反馈
- 修改密码
- 退出学生模式

2.教师

- 登录
- 查看个人信息
- 查看任教课程
- 进行成绩登记
- 修改登录密码
- 退出教师模式

3.宿舍管理员

- 登录
- 查看个人信息

- 管理宿舍信息
- 管理学生住宿信息
- 管理学生出入登记信息
- 管理学生宿舍报修信息
- 管理学生建议与反馈
- 修改登录密码
- 退出宿舍管理员模式

4.系统管理员

- 管理宿舍楼信息
- 管理宿舍信息
- 管理宿管信息
- 管理学生信息
- 管理学生住宿信息
- 进行寝室分配
- 管理教师信息
- 管理课程信息
- 删库跑路
- 退出系统管理员模式

3. 详细设计（代码）

3.1 宿舍管理员模块

DorAdminFunction

（1）查看个人信息

宿舍管理员可以查看自己的个人信息，编号，姓名，性别，管理的宿舍楼号等信息。

```
public JPanel info() { //[个人信息]功能
        String[] admin_info = new String[4]; //宿管信息
        String[] floor_info = {"无", "无", "无", "无", "无"}; //管理的宿舍楼信息
        try {
            String sql = "SELECT * FROM admin WHERE admin_num="+ admin_num; //SQL语句，查询宿管信息
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
```

```
            ps = HomePage.connection.prepareStatement(sql); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            while (rs.next()) {
                admin_info[0] = rs.getString("admin_num");
                admin_info[1] = rs.getString("name");
                admin_info[2] = rs.getString("sex");
                admin_info[3] = rs.getString("phone");
            }
            sql = "SELECT * FROM floor WHERE admin_num=" + admin_num; //SQL语句，查询该宿管管理的宿舍楼信息
            ps = HomePage.connection.prepareStatement(sql); //把操作数据库返回的结果保存到ps中
            rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            if (rs.next()) { //若该宿管有管理的宿舍楼，则获取相应宿舍楼信息
                floor_info[0] = rs.getString("floor_num");
                floor_info[1] = rs.getString("layer_amount");
                floor_info[2] = rs.getString("room_amount");
                floor_info[3] = rs.getString("category");
                floor_info[4] = rs.getString("sex");
            }
            rs.close();
            ps.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
```

（2）宿舍管理

查看全部宿舍信息，查看某个宿舍信息，将宿舍信息导出到excel表，添加宿舍信息、修改宿舍信息、删除宿舍信息。

如查看全部宿舍信息：

```
    public void allDormitory() { //该宿管管理的宿舍楼的所有宿舍
```

```
        String[] columnNames = {"楼号", "楼层", "宿舍号", "总床位数", "剩余床位数", "单价", "操作"}; //表格列名
        String[][] rowData = null; //表格数据
        int count = 0; //表的元组总数
        try { //获取dormitory表信息
            String sql = "SELECT * FROM dormitory WHERE floor_num='" + floor_num + "'"; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
            ps = HomePage.connection.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.TYPE_FORWARD_ONLY); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            rs.last();
            count = rs.getRow(); //获取宿舍元组总数
            if (count == 0) { //若dormitory表无元组
                rowData = new String[1][7];
                for (int i = 0; i < 7; i++)
                    rowData[0][i] = "无";
            } else { //若dormitory表有元组
                rowData = new String[count][7];
                rs.first();
                int i = 0;
                do { //获取该宿管管理的宿舍楼的所有宿舍信息
                    rowData[i][0] = rs.getString("floor_num"); //楼号
                    rowData[i][1] = rs.getString("layer"); //楼层
                    rowData[i][2] = rs.getString("room_num"); //宿舍号
                    rowData[i][3] = rs.getString("bed_total"); //总床位数
                    rowData[i][4] = rs.getString("bed_surplus"); //剩余床位数
                    rowData[i][5] = rs.getString("price"); //单价
                    i++;
                } while (rs.next());
            }
            rs.close();
            ps.close();
        } catch (SQLException e) {
            e.printStackTrace();
```

```
        }
```

（3）住宿管理

查看全部学生住宿信息，查询单个学生住宿信息，将住宿信息导出到excel表，修改学生住宿信息，删除学生住宿信息，添加学生住宿信息。

如修改学生住宿信息：

```
        bt_modify.addActionListener(new ActionListener() { //修改学生信息
            public void actionPerformed(ActionEvent e) {
                for (int i = 0; i < 10; i++)
                    if (i != 9)
                        tf[i].setEditable(true);
                if (x == 1) {
                    pn_first_2.remove(bt_modify);
                    pn_first_2.remove(bt_delete);
                    dorAdminJFrame.repaint();
                    pn_first_2.add(bt_save);
                    pn_first_2.add(bt_cancel);
                    dorAdminJFrame.validate();
                } else {
                    pn_second_2.remove(bt_modify);
                    pn_second_2.remove(bt_delete);
                    dorAdminJFrame.repaint();
                    pn_second_2.add(bt_save);
                    pn_second_2.add(bt_cancel);
                    dorAdminJFrame.validate();
                }
            }
        });
```

（4）出入登记

查看全部学生出入登记信息，查询单个学生出入登记信息，将出入登记信息导出到excel表，修改学生出入登记信息，删除学生出入登记信息，添加学生出入登记信息。

如添加学生出入登记信息：

```
public void addInoutInfo() { //添加出入信息
        pn_first_2.removeAll();
        pn_first_2.setLayout(null);
        pn_first_2.setBounds(0, 0, 1045, 695);

 pn_first_2.setBorder(BorderFactory.createEtchedBorder());
        JButton bt_back = new JButton("返回", new
ImageIcon("image/返回.png"));
        bt_back.setFont(new Font("黑体", 0, 17));
        bt_back.setBounds(1, 10, 92, 25);
        bt_back.setContentAreaFilled(false);
        bt_back.setBorderPainted(false);
        bt_back.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JButton bt_confirm = new JButton("确认添加", new
ImageIcon("image/confirm.png"));
        bt_confirm.setFont(new Font("黑体", 0, 17));
        bt_confirm.setBounds(447, 520, 150, 50);
        bt_confirm.setContentAreaFilled(false);
        bt_confirm.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JButton bt_time = new JButton("获取当前时间", new
ImageIcon("image/time.png"));
        bt_time.setFont(new Font("黑体", 0, 17));
        bt_time.setBounds(765, 415, 170, 40);
        bt_time.setContentAreaFilled(false);
        bt_time.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JLabel lb_infoImage = new JLabel(new
ImageIcon("image/info.png"));
        lb_infoImage.setBounds(750, 205, 200, 200);
        String[] inorout = {"--------请选择--------", "
出", "          入"};
        JComboBox<String> cb_inorout = new JComboBox<String>
(inorout);
        cb_inorout.setFont(new Font("黑体", 0, 20));
        cb_inorout.setBounds(392, 360, 260, 40);
        pn_first_2.add(bt_back);
        pn_first_2.add(bt_confirm);
        pn_first_2.add(bt_time);
        pn_first_2.add(lb_infoImage);
        pn_first_2.add(cb_inorout);
        ............
```

（5）宿舍报修

查看全部宿舍报修信息，查看单个宿舍报修信息，将所有宿舍报修信息导出到excel表，删除宿舍报修信息。

如查看全部宿舍报修信息：

```
    public void allRepair() { //该宿管管理的宿舍楼的所有宿舍报修
        String[] columnNames = {"学号", "姓名", "楼号", "楼层", "宿舍号", "报修详细", "是否处理", "操作"}; //表格列名
        String[][] rowData = null; //表格数据
        int count = 0; //表的元组总数
        try { //获取student_repair视图信息
            String sql = "SELECT * FROM student_repair WHERE floor_num='" + floor_num + "'"; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
            ps = HomePage.connection.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.TYPE_FORWARD_ONLY); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            rs.last();
            count = rs.getRow(); //获取报修信息元组总数
            if (count == 0) { //若student_repair视图无元组
                rowData = new String[1][8];
                for (int i = 0; i < 8; i++)
                    rowData[0][i] = "无";
            } else { //若student_repair视图有元组
                rowData = new String[count][8];
                rs.first();
                int i = 0;
                do { //获取该宿管管理的宿舍楼的所有报修信息
                    rowData[i][0] = rs.getString("stu_num"); //学号
                    rowData[i][1] = rs.getString("name"); //姓名
                    rowData[i][2] = rs.getString("floor_num"); //楼号
                    rowData[i][3] = rs.getString("layer"); //楼层
```

```
                    rowData[i][4] = rs.getString("room_num"); //宿舍号
                    rowData[i][5] = rs.getString("info"); //报修详细
                    if (rs.getString("yes_no").equals("是"))
                        rowData[i][6] = "<html><font color='green'>是</font></html>";
                    else
                        rowData[i][6] = "<html><font color='red'>否</font></html>";
                    i++;
                } while (rs.next());
            }
            rs.close();
            ps.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
```

（6）建议与反馈

查看所有学生建议与反馈，查看某个学生建议与反馈,将所有建议与反馈导出到excel表，删除学生建议与反馈。

如查看某个学生建议与反馈：

```
    public void visitAdviceInfo(String stu_num, String info) {//查看建议与反馈信息，可删除
        pn_first_2.removeAll();
        pn_first_2.setLayout(null);
        pn_first_2.setBounds(0, 0, 1045, 735);

 pn_first_2.setBorder(BorderFactory.createEtchedBorder());
        JButton bt_back = new JButton("返回", new ImageIcon("image/返回.png"));
        bt_back.setFont(new Font("黑体", 0, 17));
        bt_back.setBounds(1, 10, 92, 25);
        bt_back.setContentAreaFilled(false);
        bt_back.setBorderPainted(false);
```

```
        bt_back.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JButton bt_delete = new JButton("删除建议与反馈", new
ImageIcon("image/delete.png"));
        bt_delete.setFont(new Font("黑体", 0, 17));
        bt_delete.setBounds(805, 410, 190, 40);
        bt_delete.setForeground(Color.red);
        bt_delete.setContentAreaFilled(false);
        bt_delete.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JLabel lb_infoImage = new JLabel(new ImageIcon("image/建
议与反馈.png"));
        lb_infoImage.setBounds(800, 200, 200, 200);
        pn_first_2.add(bt_back);
        pn_first_2.add(bt_delete);
        pn_first_2.add(lb_infoImage);
```

（7）修改密码

修改自身管理员密码

```
    public JPanel changePassword() { //[修改密码]功能
        JPanel pn_changePassword = new JPanel();
        pn_changePassword.setLayout(null);
        pn_changePassword.setSize(1045, 735);

pn_changePassword.setBorder(BorderFactory.createEtchedBorder());
        JLabel lb_old = new JLabel("旧密码："), lb_new1 = new
JLabel("新密码："), lb_new2 = new JLabel("确认密码："), lb = new
JLabel("（密码不超过20位）");
        JPasswordField pf_old = new JPasswordField(), pf_new1 =
new JPasswordField(), pf_new2 = new JPasswordField();
        .......
```

（8）退出

退出宿管模式。

```
        if (lb_tips.getText().equals("是否退出宿管模式？")) { //确定退出宿管模式，返回到欢迎界面
            try {
                HomePage.connection.close(); //关闭数据库连接
            } catch (SQLException e) {
                e.printStackTrace();
            }
            dorAdminJFrame.dispose();
            HomePage.con.removeAll();
            HomePage.mainJFrame.repaint();
            HomePage.welcomePage();
            HomePage.mainJFrame.validate();
            ......
```

3.2 学生模块

StudentFunction

（1）个人信息

查看学生个人信息，如学号，姓名，性别，院系，班级等

```
private void showInfo() {
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(info());
        studentJFrame.validate();
    }
```

（2）选课

选修某门课程并且能查看已选所有课程，将选课记录导出到excel表。

```
private void showSelectCourse() {
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(select_course());
        studentJFrame.validate();
    }
```

（3）我的课程

查看学生的选课情况及课程分数，将我的课程记录导出到excel表。

```
 private void showMyCourse() {
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(my_course());
        studentJFrame.validate();
    }
```

（4）宿舍报修

提交宿舍报修申请。

```
private void showRepair() {
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(repair());
        studentJFrame.validate();
    }
```

（5）宿舍反馈

提交对宿舍的建议与反馈。

```
private void showAdvise() {
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(advise());
        studentJFrame.validate();
    }
```

（6）修改密码

修改学生登录密码。

```
private void showChangePassword() {
        //lb2.setText("[修改密码]");
        pn_function.removeAll();
        studentJFrame.repaint();
        pn_function.add(changePassword());
        studentJFrame.validate();
    }
```

（7）退出

退出学生模式。

```
private void showExitConfirmation() {
        lb_tips.setText("是否退出学生模式？");
        choiceTips();
    }
```

3.3 教师模块

TeacherFunction

（1）个人信息

查看教师个人信息，如工号，姓名，性别等。

```
public JPanel info() { //[个人信息]功能
        String[] teacher_info = new String[3]; //教师信息
        Calendar cal = Calendar.getInstance();
        try {
            String sql = "SELECT * FROM teacher WHERE tno=" +
teacher_num; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象
ps，用来执行SQL语句
```

```
            ps = HomePage.connection.prepareStatement(sql); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            while (rs.next()) {
                teacher_info[0] = rs.getString("tno");
                teacher_info[1] = rs.getString("tname");
                teacher_info[2] = rs.getString("tsex");
            }
            rs.close();
            ps.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
        .......
```

（2）任教课程

查看教师任教课程，将任教课程导出到excel表。

```
 JPanel tp_mycourse = new JPanel();
        JPanel tp_mycourse_1 = new JPanel();
        tp_mycourse.setLayout(null);
        tp_mycourse_1.setLayout(null);

        tp_mycourse.setBounds(0, 0, 1045, 735);
        tp_mycourse_1.setBounds(0, 0, 1045, 50);
        String[] columnNames = {"课程号", "课程名", "选课人数"}; //表格列名
        String[][] rowData = null; //表格数据
        int count = 0; //表的元组总数
        try { //获取course表信息
            String sql = "SELECT cno,cname FROM course WHERE tno=" + teacher_num; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
            ps = HomePage.connection.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.TYPE_FORWARD_ONLY); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
```

```
            rs.last();
            count = rs.getRow(); //获取宿舍元组总数
            if (count == 0) { //若dormitory表无元组
                rowData = new String[1][3];
                for (int i = 0; i < 3; i++)
                    rowData[0][i] = "无";
            } else { //若dormitory表有元组
            ......
```

（3）成绩登记

登记某个学生所授课程的成绩，将课程成绩记录导入到excel表。

```
JPanel tp_mycourse = new JPanel();
        JPanel tp_mycourse_1 = new JPanel();
        tp_mycourse.setLayout(null);
        tp_mycourse_1.setLayout(null);

        tp_mycourse.setBounds(0, 0, 1045, 735);
        tp_mycourse_1.setBounds(0, 0, 1045, 50);
        String[] columnNames = {"课程号", "课程名", "学生号", "学生名", "成绩", "操作"}; //表格列名
        String[][] rowData = null; //表格数据
        int count = 0; //表的元组总数
        try { //获取course表信息
            String sql = "SELECT course.cno,cname,stu_num,name,mark FROM course,select_course,student WHERE course.cno=select_course.cno && stu_num=sno && course.tno=" + teacher_num; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
            ps = HomePage.connection.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.TYPE_FORWARD_ONLY); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            rs.last();
            count = rs.getRow(); //获取元组总数
            if (count == 0) { //若dormitory表无元组
                rowData = new String[1][6];
                for (int i = 0; i < 6; i++)
```

```
21                      rowData[0][i] = "无";
```

（4）修改密码

修改教师登录密码。

```
1  public JPanel changePassword() { //[修改密码]功能
2          JPanel pn_changePassword = new JPanel();
3          pn_changePassword.setLayout(null);
4          pn_changePassword.setSize(1045, 735);
5
    pn_changePassword.setBorder(BorderFactory.createEtchedBorder())
   ;
6          JLabel lb_old = new JLabel("旧密码："), lb_new1 = new
   JLabel("新密码："), lb_new2 = new JLabel("确认密码："), lb = new
   JLabel("（密码不超过20位）");
7          JPasswordField pf_old = new JPasswordField(), pf_new1 =
   new JPasswordField(), pf_new2 = new JPasswordField();
8          ......
9          bt_reset.addActionListener(new ActionListener() { //重
   置，将密码框内容清空
10             public void actionPerformed(ActionEvent e) {
11                 pf_old.setText("");
12                 pf_new1.setText("");
13                 pf_new2.setText("");
14             }
15         });
16         bt_confirm.addActionListener(new ActionListener() { //确
   认xxxxxxxxxx public JPanel changePassword()
17         ......
```

（5）退出

退出教师模式。

```
public void yesOperation() { //选择“是”之后，根据提示内容，选择要进行的操作
        if (lb_tips.getText().equals("是否退出教师模式？")) { //退出教师模式，返回到欢迎界面

            teacherJFrame.dispose();
            returnHomepage();
        }
    }
}
```

3.4 系统管理员模块

SysAdminFunction

（1）学生管理

查看全部学生信息，查看某个学生信息，将学生信息导出到excel表，添加学生信息、修改学生信息、删除学生信息。

```
    public JTabbedPane studentManage() { //学生管理
        JTabbedPane tp_student = new JTabbedPane();
        tp_student.setFont(new Font("黑体", 0, 25));
        tp_student.setBounds(0, 0, 1045, 735);
        allStudent();
        tp_student.addTab(" 全部 ", pn_first);
        queryStudentInfo_1();
        tp_student.addTab(" 查询 ", pn_second);
        return tp_student;
    }

    //查看所有学生信息
    public void allStudent()

    //查看单个学生信息并进行修改/删除
    public void visitStudentInfo(String stu_num, int x, String query_sql)

```

```
18      //添加学生信息
19      public void addStudentInfo()
20
21      //查询学生信息
22      public void queryStudentInfo_1()
23
24      //查询到学生信息的表
25      public void queryStudentInfo_2(String sql)
```

（2）教师管理

查看全部教师信息，查看某个教师信息，将教师信息导出到excel表，添加教师信息、修改教师信息、删除教师信息。

```
1       public JTabbedPane teacherManage() { //教师管理
2           JTabbedPane tp_teacher = new JTabbedPane();
3           tp_teacher.setFont(new Font("黑体", 0, 25));
4           tp_teacher.setBounds(0, 0, 1045, 735);
5           allTeacher();
6           tp_teacher.addTab(" 全部 ", pn_first);
7           queryTeacherInfo_1();
8           tp_teacher.addTab(" 查询 ", pn_second);
9           return tp_teacher;
10      }
11
12      public void allTeacher()
13
14      public void visitTeacherInfo(String tno_num, int x, String
    query_sql)
15
16      public void addTeacherInfo()
17
18      public void queryTeacherInfo_1()
19
20      public void queryTeacherInfo_2(String sql)
```

（3）课程管理

查看全部课程信息，查看某个课程信息，将课程信息导出到excel表，添加课程信息、修改课程信息、删除课程信息。

```
    public JTabbedPane courseManage() { //课程管理
        JTabbedPane tp_course = new JTabbedPane();
        tp_course.setFont(new Font("黑体", 0, 25));
        tp_course.setBounds(0, 0, 1045, 735);
        allCourse();
        tp_course.addTab(" 全部 ", pn_first);
        queryCourseInfo_1();
        tp_course.addTab(" 查询 ", pn_second);
        return tp_course;
    }

    public void allCourse()

    public void visitCourseInfo(String tno_num, int x, String
query_sql)

    public void addCourseInfo()

    public void queryCourseInfo_1()

    public void queryCourseInfo_2(String sql)
```

（4）选课管理

查看全部学生选课信息，查看某条学生选课信息，将学生选课信息导出到excel表，添加学生选课信息、修改学生选课信息、删除学生选课信息。

```
    private JTabbedPane select_course_Manage() {
        JTabbedPane tp_teacher = new JTabbedPane();
        tp_teacher.setFont(new Font("黑体", 0, 25));
        tp_teacher.setBounds(0, 0, 1045, 735);
        all_select_course();
        tp_teacher.addTab(" 全部 ", pn_first);
        querySelectCourse();
        //tp_teacher.addTab(" 查询 ", pn_second);
        return tp_teacher;
    }

    //查看选课
    private void querySelectCourse()

```

```
    //查看选课信息
    private void querySelectCourse_1(String sql)

    //退选
    private void unselect(String cno, String sno)

    //判定是否退选
    private void unselect_course(String cno, String sno)
```

（5）宿管管理

查看全部宿管信息，查看某个宿管信息，将宿管信息导出到excel表，添加宿管信息、修改宿管信息、删除宿管信息。

```
    public JTabbedPane adminManage() { //宿管管理
        JTabbedPane tp_admin = new JTabbedPane();
        tp_admin.setFont(new Font("黑体", 0, 25));
        tp_admin.setBounds(0, 0, 1045, 735);
        allAdmin();
        tp_admin.addTab(" 全部 ", pn_first);
        queryAdminInfo_1();
        tp_admin.addTab(" 查询 ", pn_second);
        return tp_admin;
    }
    public void allAdmin()

    public void visitAdminInfo(String admin_num, int x, String query_sql)

    public void addAdminInfo()

    public void queryAdminInfo_1()

    public void queryAdminInfo_2(String sql)

```

（6）宿舍管理

查看全部宿舍信息，查看某个宿舍选课信息，将宿舍信息导出到excel表，添加宿舍信息、修改宿舍信息、删除宿舍信息。

```
    public JTabbedPane dormitoryManage() { //宿舍管理
        JTabbedPane tp_dormitory = new JTabbedPane();
        tp_dormitory.setFont(new Font("黑体", 0, 25));
        tp_dormitory.setBounds(0, 0, 1045, 735);
        allDormitory();
        tp_dormitory.addTab(" 全部 ", pn_first);
        queryDormitoryInfo_1();
        tp_dormitory.addTab(" 查询 ", pn_second);
        return tp_dormitory;
    }

    public void allDormitory()

    public void visitDormitoryInfo(String floor_num, String
layer, String room_num, int x, String query_sql)

    public void addDormitoryInfo()

    public void queryDormitoryInfo_1()

    public void queryDormitoryInfo_2()
```

（7）宿舍楼管理

查看全部宿舍楼信息，查看某个宿舍楼信息，将宿舍楼信息导出到excel表，添加宿舍楼信息、修改宿舍楼信息、删除宿舍楼信息。

```
    public JTabbedPane floorManage() { //宿舍楼管理
        JTabbedPane tp_floor = new JTabbedPane();
        tp_floor.setFont(new Font("黑体", 0, 25));
        tp_floor.setBounds(0, 0, 1045, 735);
        allFloor();
        tp_floor.addTab(" 全部 ", pn_first);
        queryFloorInfo_1();
        tp_floor.addTab(" 查询 ", pn_second);
        return tp_floor;
    }

    public void allFloor()

```

```
14      public void visitFloorInfo(String floor_num, int x, String
    query_sql)
15
16      public void addFloorInfo()
17
18      public void queryFloorInfo_1()
19
20      public void queryDormitoryInfo_2()
```

（8）住宿管理

查看全部住宿信息，查看某条住宿信息，将住宿信息导出到excel表，添加住宿信息、修改住宿信息、删除住宿信息。

```
1       public JTabbedPane stayManage() { //住宿管理
2           JTabbedPane tp_stay = new JTabbedPane();
3           tp_stay.setFont(new Font("黑体", 0, 25));
4           tp_stay.setBounds(0, 0, 1045, 735);
5           allStay();
6           tp_stay.addTab(" 全部 ", pn_first);
7           queryStayInfo_1();
8           tp_stay.addTab(" 查询 ", pn_second);
9           return tp_stay;
10      }
11
12      public void allStay()
13
14      public void visitStayInfo(String stu_num, int x, String
    query_sql)
15
16      public void addStayInfo()
17
18      public void queryStayInfo_1()
19
20      public void queryStayInfo_2(String sql)
```

（9）寝室分配

一键自动分配，一键重新分配。

```
1       //界面设置
```

```
    public JPanel allocation() { //寝室分配
        JPanel pn_allocation = new JPanel();
        pn_allocation.setLayout(null);
        pn_allocation.setSize(1045, 735);

 pn_allocation.setBorder(BorderFactory.createEtchedBorder());
        JLabel lb_dorImage = new JLabel(new ImageIcon("image/宿
舍.png"));

        pn_allocation.add(lb_dorImage);
        pn_allocation.add(lb);
        pn_allocation.add(bt1);
        pn_allocation.add(bt2);
        ......
        ......
        ......
        return pn_allocation;
    }

    //自动分配宿舍
    public void automaticallyAllocate() throws SQLException

```

（10）删库跑路

删除系统数据库。

```
    public JPanel escape() { //删库跑路
        JPanel pn_escape = new JPanel();
        pn_escape.setLayout(null);
        pn_escape.setSize(1045, 735);
        pn_escape.setBorder(BorderFactory.createEtchedBorder());
        JLabel lb_runImage = new JLabel(new ImageIcon("image/
跑.png"));
        ......
        bt_confirm.addActionListener(new ActionListener() { //确
认删库跑路
            public void actionPerformed(ActionEvent e) {
                if
(String.valueOf(pf.getPassword()).equals("ESCAPE")) { //密码正确
                    lb_tips.setText("是否确定删库跑路？");

```

```
                    choiceTips("");
                } else { //密码错误
                    lb_tips.setText("密码错误！");
                    functionTips();
                }
            }
        });
        return pn_escape;
    }

    //跑路弹窗
    public void functionTips()
```

（11）退出

退出系统管理员模式。

```
        if (e.getActionCommand().equals("退出")) {
            lb_tips.setText("是否退出系统管理模式？");
            choiceTips("");
        }
```

3.5 主页模块

HomePage

欢迎界面，更改外观选项，四个登录选项：学生登录，宿管登录，教师登录，系统管理员登录。

如四个登录选项：

```
private static void performLogin(String username, String password) {
        switch (userType) {
            case 1 -> new Login().studentLogin(username, password);
            case 2 -> new Login().dor_adminLogin(username, password);
            case 3 -> new Login().teacherLogin(username, password);
            case 4 -> new Login().adminLogin(username, password);
            default -> System.out.print("Unknown user type");
        }
    }
```

3.6 登录模块

Login

四种不同的身份登录：学生登录，宿管登录，教师登录，系统管理员登录

如学生登录：

```
    public void studentLogin(String account, String password) { //学生登录
        boolean flag = false; //是否查找到相应账号密码
        String stu_num = null, pass; //学生账号密码
        try {
            String sql = "SELECT stu_num, password FROM student"; //SQL语句
            PreparedStatement ps; //创建PreparedStatement类对象ps，用来执行SQL语句
            ps = HomePage.connection.prepareStatement(sql); //把操作数据库返回的结果保存到ps中
            ResultSet rs = ps.executeQuery(sql); //ResultSet类，用来存放获取的结果集
            while (rs.next()) { //遍历结果集
                stu_num = rs.getString("stu_num");
                pass = rs.getString("password");
                if (account.equals(stu_num) && password.equals(pass)) {
                    flag = true;
```

```
                    break;
                }
            }
            rs.close();
            ps.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
        if (flag) { //若学生账号密码正确，则进入学生功能界面
            lb_tips.setText("登录成功！");
            loginTips();
            new StudentFunction(stu_num);
            HomePage.mainJFrame.dispose();
        } else { //若账号密码错误，则进行提示
            lb_tips.setText("账号或密码错误！");
            loginTips();
            try {
                HomePage.connection.close(); //关闭数据库连接
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
```

3.7 各类信息模块

info

定义各种类，用于存储所有信息

如学生类：

```
class Student { //学生
    String stu_num; //学号
    String password; //密码
    String name; //姓名
    String sex; //性别
    int birth; //出生日期
```

```
    int grade; //年级
    String faculty; //院系
    String clas; //班级
    String phone; //手机号
    String yes_no; //是否入住

    public Student(String stu_num, String password, String name, String sex, String birth, String grade, String faculty, String clas, String phone, String yes_no) {
        this.stu_num = stu_num;
        this.password = password;
        this.name = name;
        this.sex = sex;
        this.birth = Integer.parseInt(birth);
        this.grade = Integer.parseInt(grade);
        this.faculty = faculty;
        this.clas = clas;
        this.phone = phone;
        this.yes_no = yes_no;
    }
}
........
```

宿管类：

```
class Admin { //宿管
    String admin_num; //编号
    String password; //密码
    String name; //姓名
    String sex; //性别
    String phone; //手机

    public Admin(String admin_num, String password, String name, String sex, String phone) {
        this.admin_num = admin_num;
        this.password = password;
        this.name = name;
        this.sex = sex;
        this.phone = phone;
    }
}
```

3.8 导出表模块

JTableToExcel

用于将信息导出到excel表

```
public class JTableToExcel { //用于将表格导出成Excel文件
    public static void export(File file, JTable table) { //导出方法
        try {
            WritableWorkbook workbook = null; //创建工作薄
            if (file.exists()) { //文件已经存在
                workbook = Workbook.createWorkbook(file, Workbook.getWorkbook(file));
            } else { //文件还不存在
                workbook = Workbook.createWorkbook(file);
            }
            // 创建工作表
            WritableSheet sheet = workbook.createSheet("Sheet1", workbook.getNumberOfSheets());
            // 取得table的行数(rowNum), 列数(colNum)
            int rowNum = table.getRowCount();
            int colNum = table.getColumnCount() - 1;
            // 填写列名
            fillColumnName(sheet, table, colNum);
            // 填写数据
            fillCell(sheet, table, rowNum, colNum);
            // 写入工作表
            workbook.write();
            workbook.close();
            // 导出成功提示框
            int dialog = JOptionPane.showConfirmDialog(null, "表格导出成功！是否现在打开？", "提示", JOptionPane.YES_NO_OPTION);
            if (dialog == JOptionPane.YES_OPTION) { //打开Excel文件
                Runtime.getRuntime().exec("cmd /c start \"\" \"" + file + "\"");
            }
        } catch (FileNotFoundException e) {
            JOptionPane.showMessageDialog(null, "导出数据前请关闭工作表！");
        } catch (Exception e) {
```

```
            JOptionPane.showMessageDialog(null, "导出失败！");
        }
    }
    ............
```

3.9 自定义功能模块

MyButton，MyButtonEditor，MyButtonRender，MyEvent ， MyTableModel，NumLimit

自定义按钮类，编辑器，表格渲染器，表格模型，文本框内容限制等一系列功能。

如：自定义的按钮类，按钮存放了其所在的行列数

```
public class MyButton extends JButton { //自定义的按钮类，按钮存放了其所在的行列数
    private int row;
    private int column;

    public MyButton() {

    }
```

自定义的表格渲染器，用于更改外观

```
public class MyButtonRender implements TableCellRenderer { //自定义的表格渲染器
    private final JButton bt;

    public MyButtonRender(String operation) {
        bt = new JButton(operation);
        bt.setFont(new Font("黑体", 0, 20));
        bt.setForeground(Color.blue);
        bt.setContentAreaFilled(false);
        bt.setBorderPainted(false);
        bt.setCursor(new Cursor(Cursor.HAND_CURSOR));
    }

```

```
13      public Component getTableCellRendererComponent(JTable table,
    Object value, boolean isSelected, boolean hasFocus, int row, int
    column) {
14          return bt;
15      }
16  }
```

4. 界面设计（用户说明）

4.1 主页

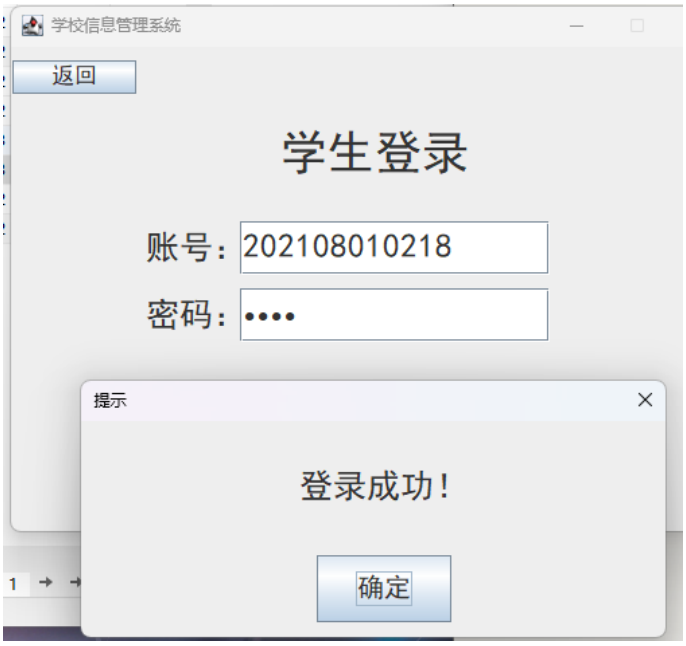
运行代码，可以看到主页上发出现欢迎信息，以及四个不同身份的登录按钮，点击不同的按钮即可进入不同的用户模式。



4.2 学生端

1.登录

输入账号，密码，点击'登录'



2.个人信息

点击'个人信息'，即可查看学生的各项信息，如学号，性别，年级，院系，班级及宿舍信息等。



3.选课

点击'选课'，即可查看全部未选课程。



点击某门课程的'选课'按钮，再点击'是'，即可选修该课程。



点击'查询'，即可查询到所有课程（包括已经选修的课程）



点击自己未选的课程进行'选课'即可选修，若点击已选修的课程进行选课，则会选课失败



点击导出，即可将所有的课程信息导出为excel表

4.我的课程

点击'我的课程'，可以查看自己选修的课程及分数



点击'导出'，即可将所有选修的课程信息导出为excel表

5.宿舍报修

点击'宿舍报修'，输入自己的宿舍信息以及报修详细，即可进行宿舍报修



6.宿舍反馈

点击'宿舍反馈'，可以输入自己的建议与反馈并提交



7.修改密码

点击'修改密码'，可以修改自己的登录密码。



8.退出

点击'退出''，再点击'是'，即可退出学生模式。



4.3 教师端

1.登录

输入账号，密码，点击'登录'



2.个人信息

点击'个人信息'，即可查看自己的信息，如工号，姓名，性别等



3.任教课程

点击'任教课程'，即可查看自己所教的课程信息（含课程号、课程名、选课人数）



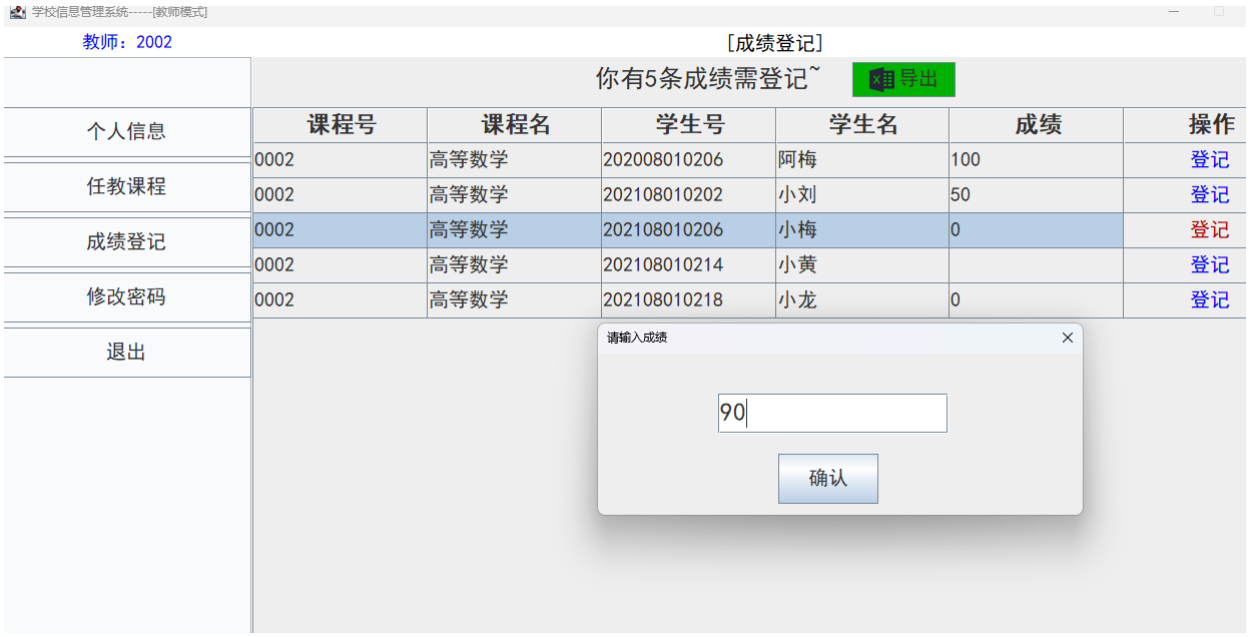
点击'导出'，即可将自己的任教课程信息导出到excel表

4.成绩登记

点击'成绩登记'，即可对所有选修所有课程的学生进行成绩登记



点击'登记'，输入分数即可



点击'导出'，即可将所有该门课程的成绩信息导出到excel表

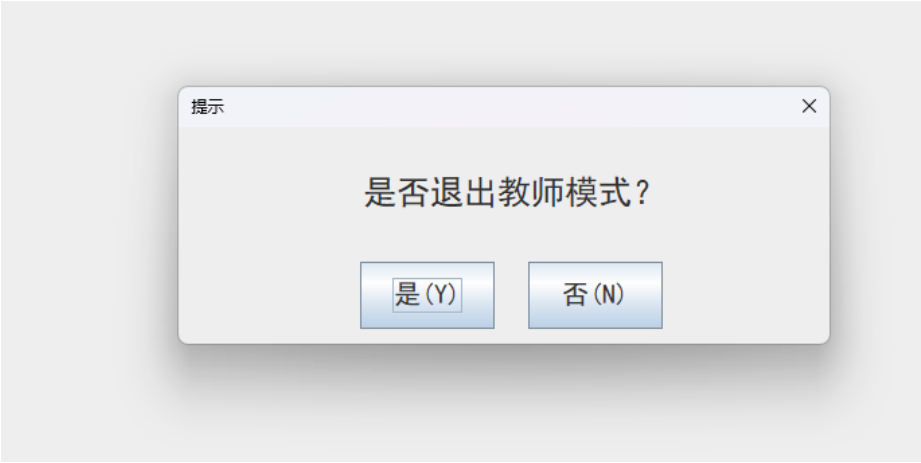
5.修改密码

点击'修改密码'，可以修改自己的登录密码或重置为初始密码。



6.退出

点击'退出'，再点击'是'，即可退出教师模式。



4.4 宿舍管理员端

1.登录

输入账号，密码，点击'登录'



2.个人信息

点击'个人信息'，可以查看自己的各项信息。

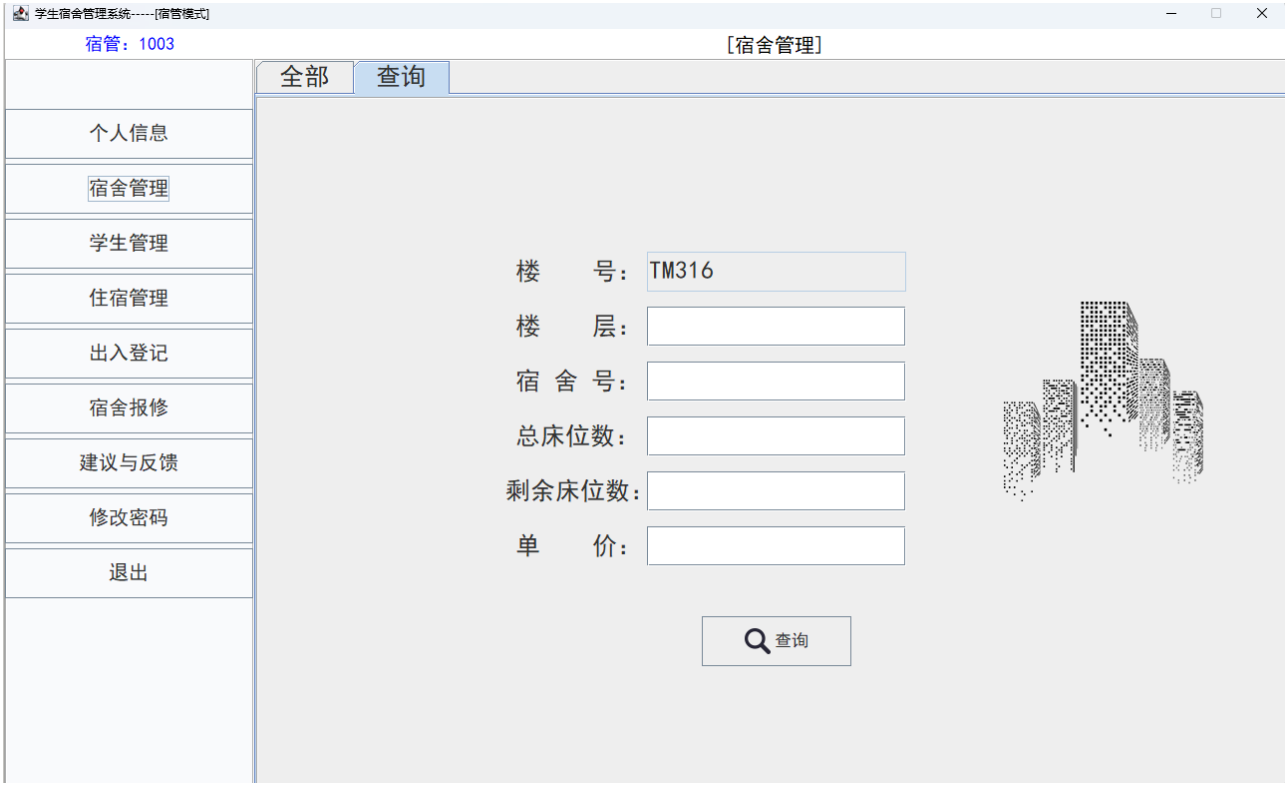


3.宿舍管理

点击'宿舍管理'，可以看到全部的宿舍信息。



点击'查询'，输入相应的楼号可以查询到该宿舍楼的信息。



点击'添加'，输入宿舍信息，可以实现添加宿舍功能。



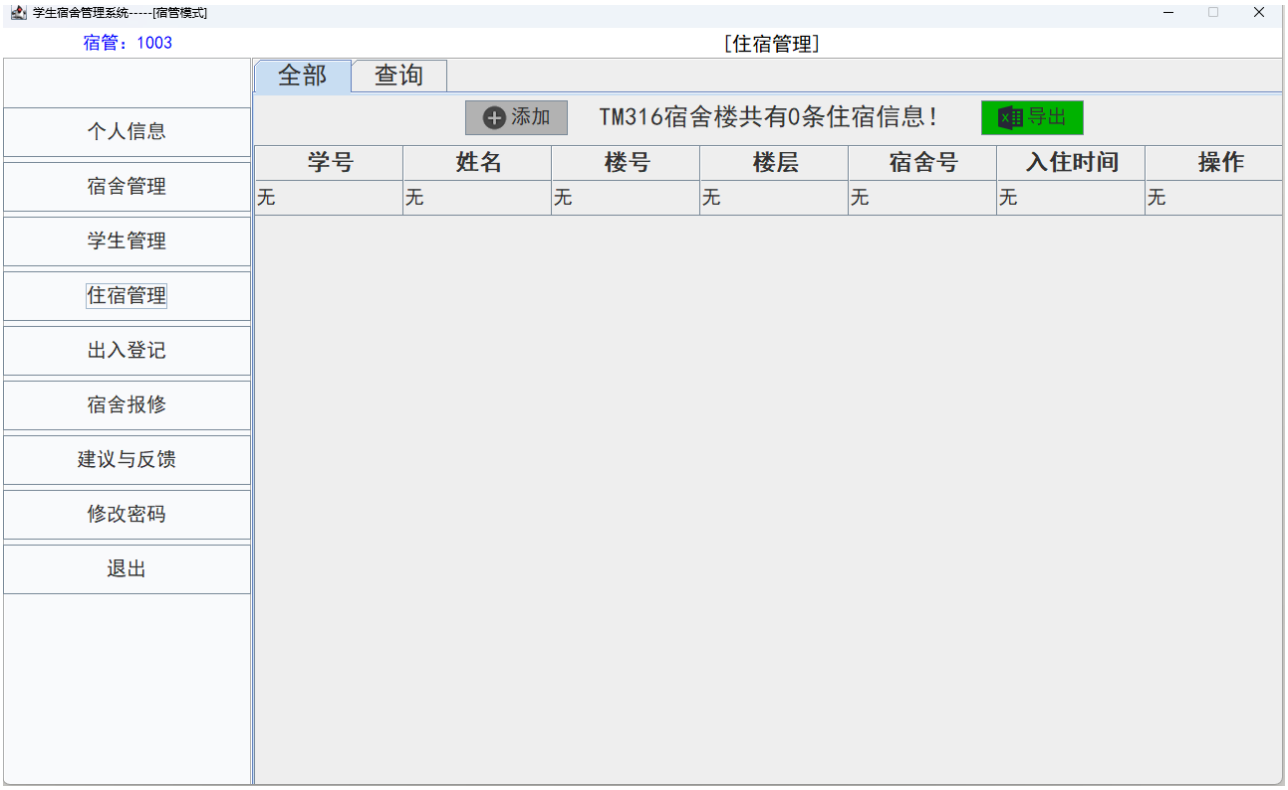
对某个宿舍点击'查'看，再点击'修改宿舍信息'，可以对宿舍信息进行修改。点击删除宿舍信息，可以对该宿舍进行删除。



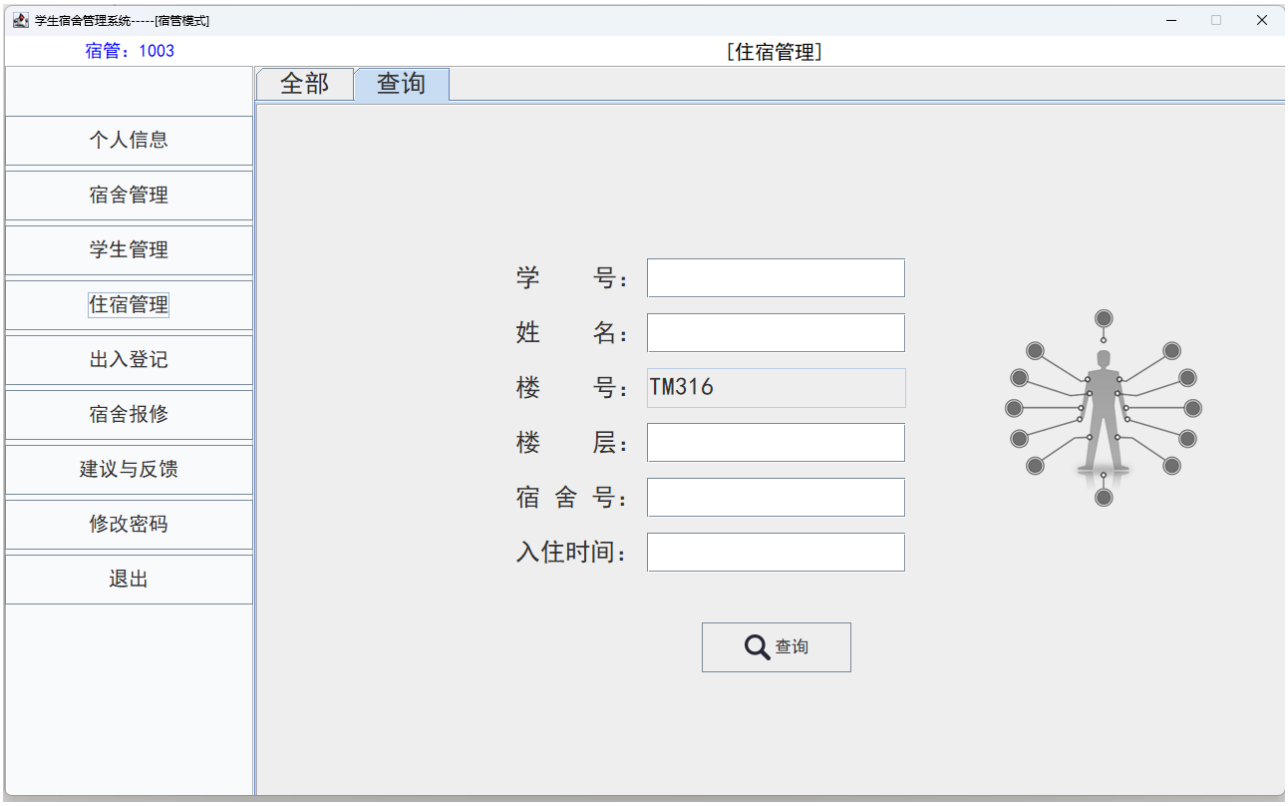
点击'导出'，即可将所有宿舍信息导出为excel表

4.住宿管理

点击'住宿管理'，可以查看所有学生的住宿信息。



点击'查询'，输入学号，可以查询该学生的住宿信息。

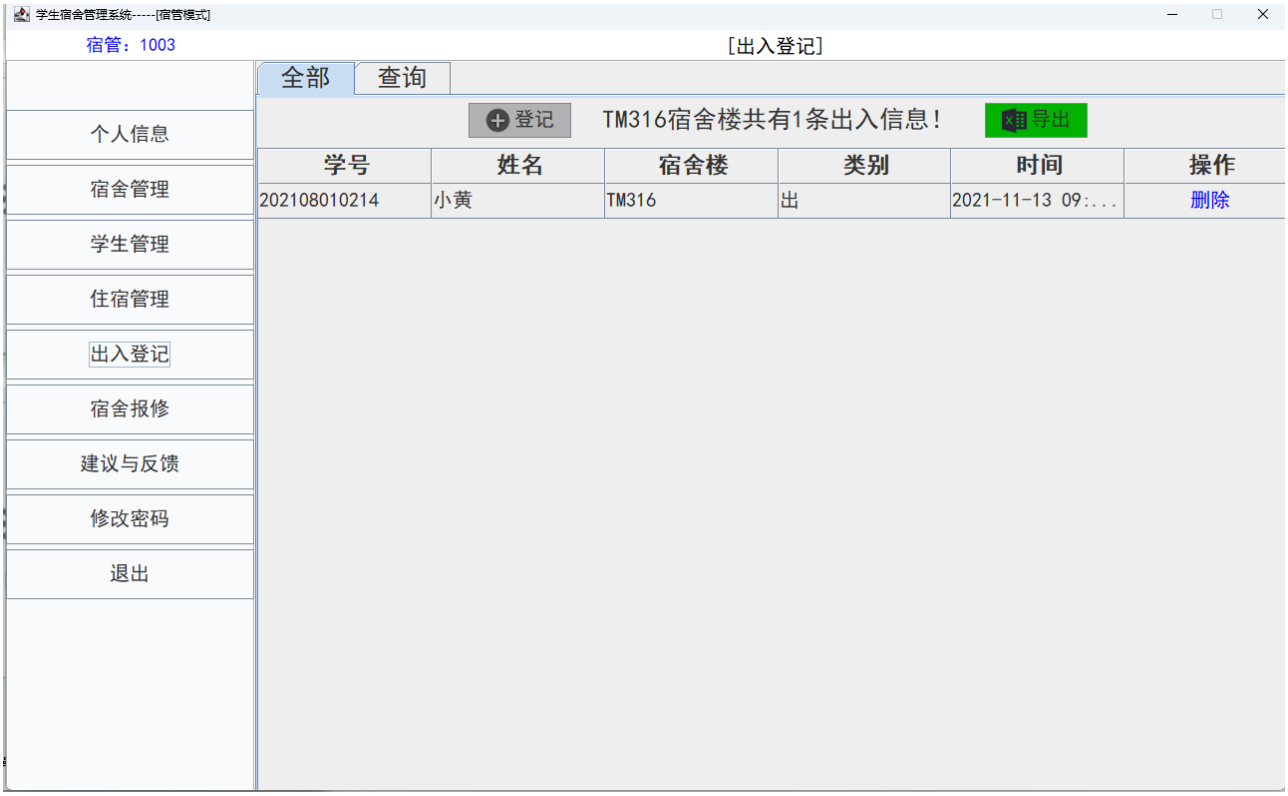


点击'添加'，可以添加指定学生的住宿信息。



5.出入登记

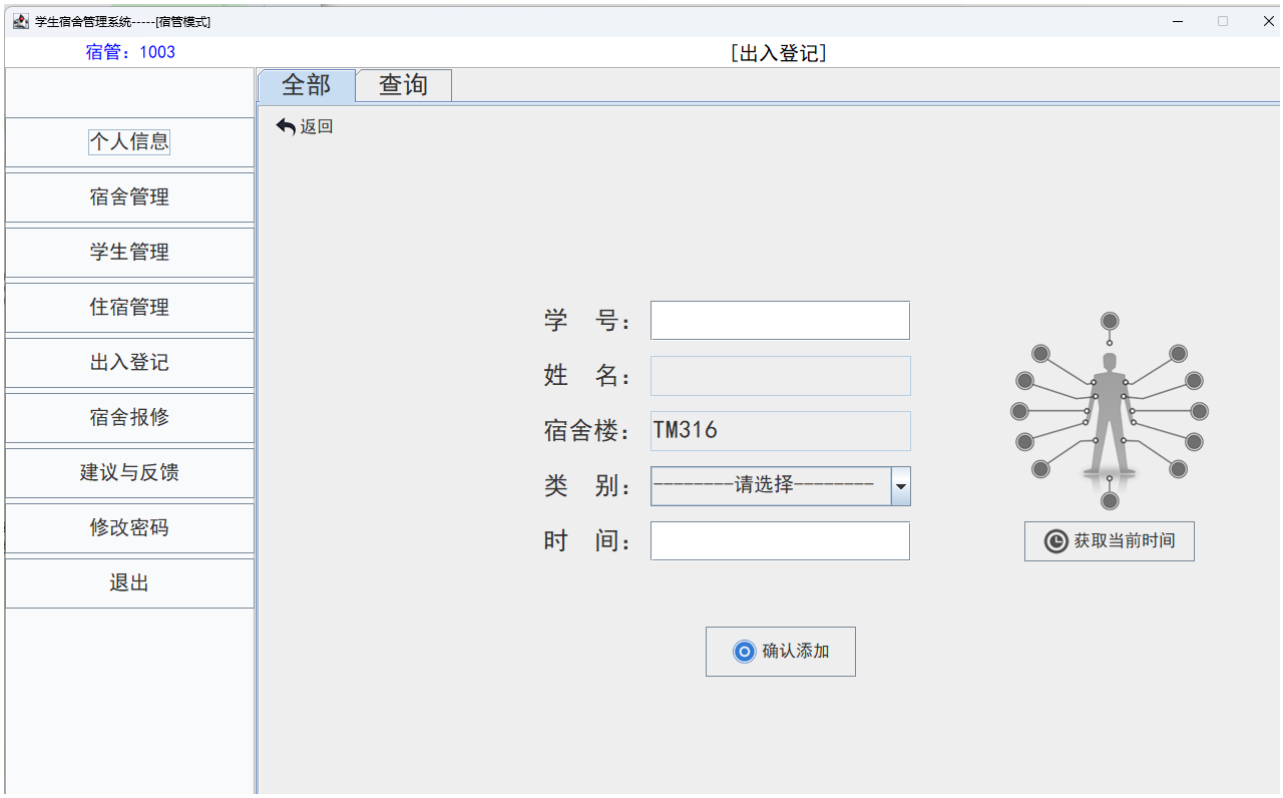
点击'出入登记'，可以看到所有出入登记信息。



点击'查询'，输入学号，可以看到该名学生的出入信息。



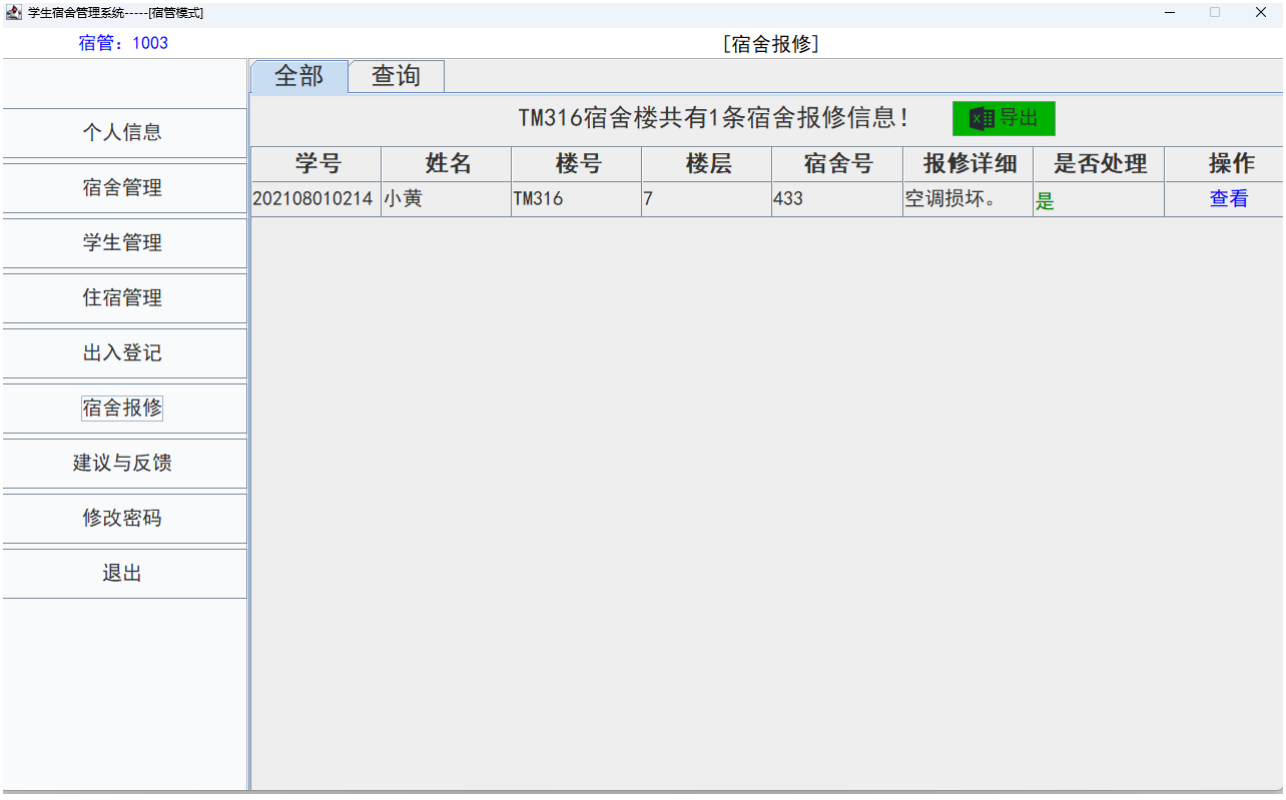
点击'添加'，可以添加指定学号的学生的出入登记信息。



点击'导出'，即可将所有出入登记信息导出到excel表

6.宿舍报修

点击'宿舍报修'，可以看到所有宿舍报修信息。



点击'查询'，输入学号，可以看到该名学生提交的宿舍报修信息。



对某条宿舍报修信息点击'查看'，可以对其进行处理和删除。



点击'导出'，即可将所有宿舍报修信息导出到excel表

7.建议与反馈

点击'建议与反馈'，可以看到所有建议与反馈信息。



点击'导出'，即可将所有建议与反馈导出到excel表

8.修改密码

点击'修改密码'，可以修改自己的登录密码。



9.退出

点击'退出'，再点击是，即可退出宿管模式。



4.5 系统管理员端

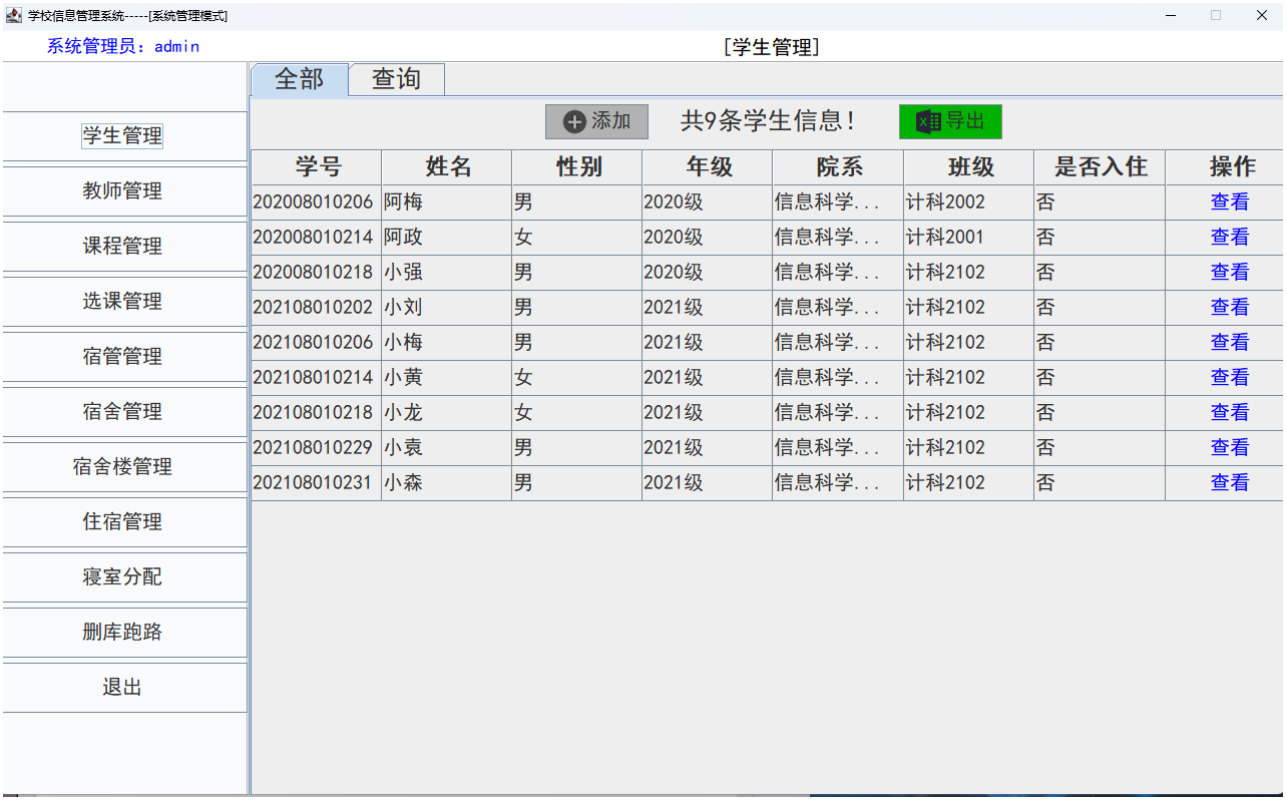
1. 登录

输入账号，密码，点击'登录'



2.学生管理

点击'学生管理'，接口查看所有学生信息



点击'添加'，可以添加单个学生信息



点击'查询'，可以查询指定学号的学生



点击某个学生的'查看'按钮，可以查看该学生的信息，并且可以点击'修改学生信息'或'删除学生信息'按钮修改和删除该学生的信息



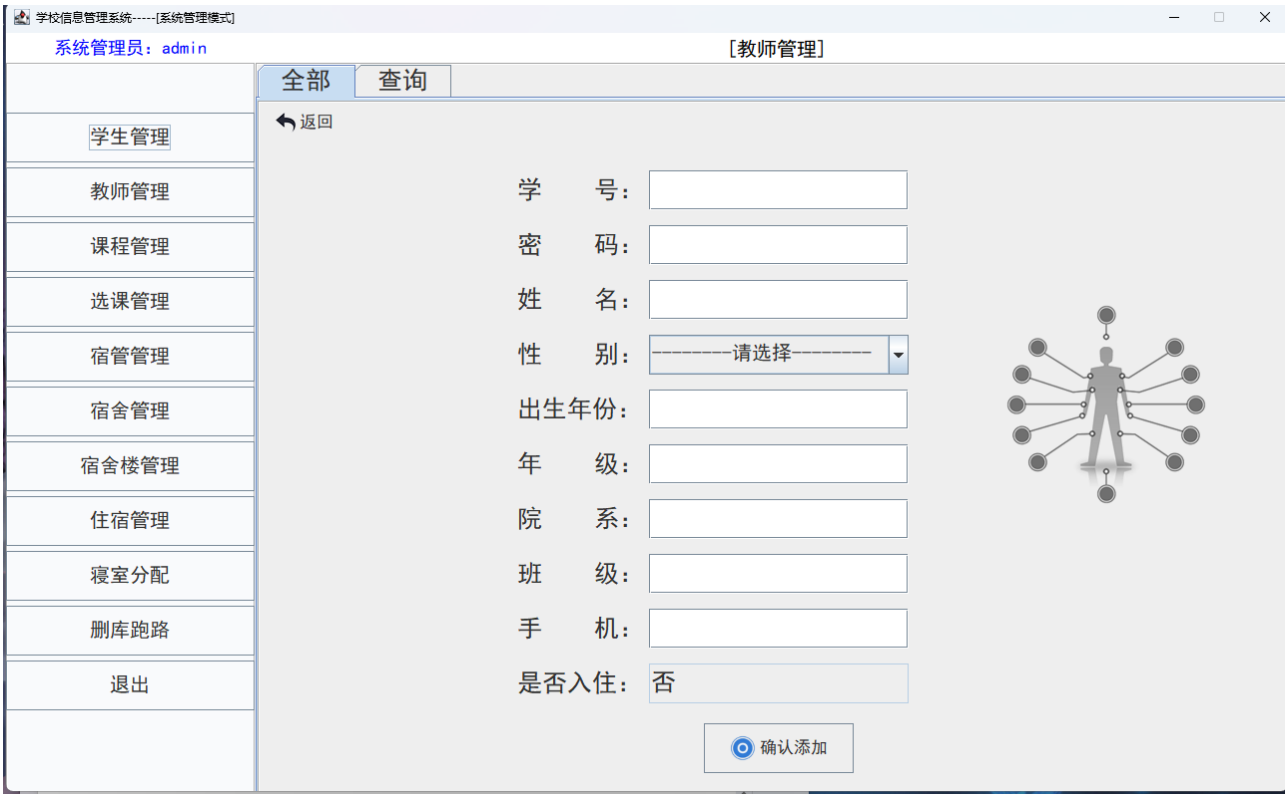
点击'导出'，即可将所有学生信息导出到excel表

3.教师管理

点击'教师管理'，可以查看所有教师的信息



点击'添加'，即可输入教师信息进行添加一名教师



点击'查询'，可以查看指定的教师信息



点击某个教师的'查看'按钮，可以查看该学生的信息，并且可以点击'修改教师信息'或'删除教师信息'按钮修改和删除该学生的信息



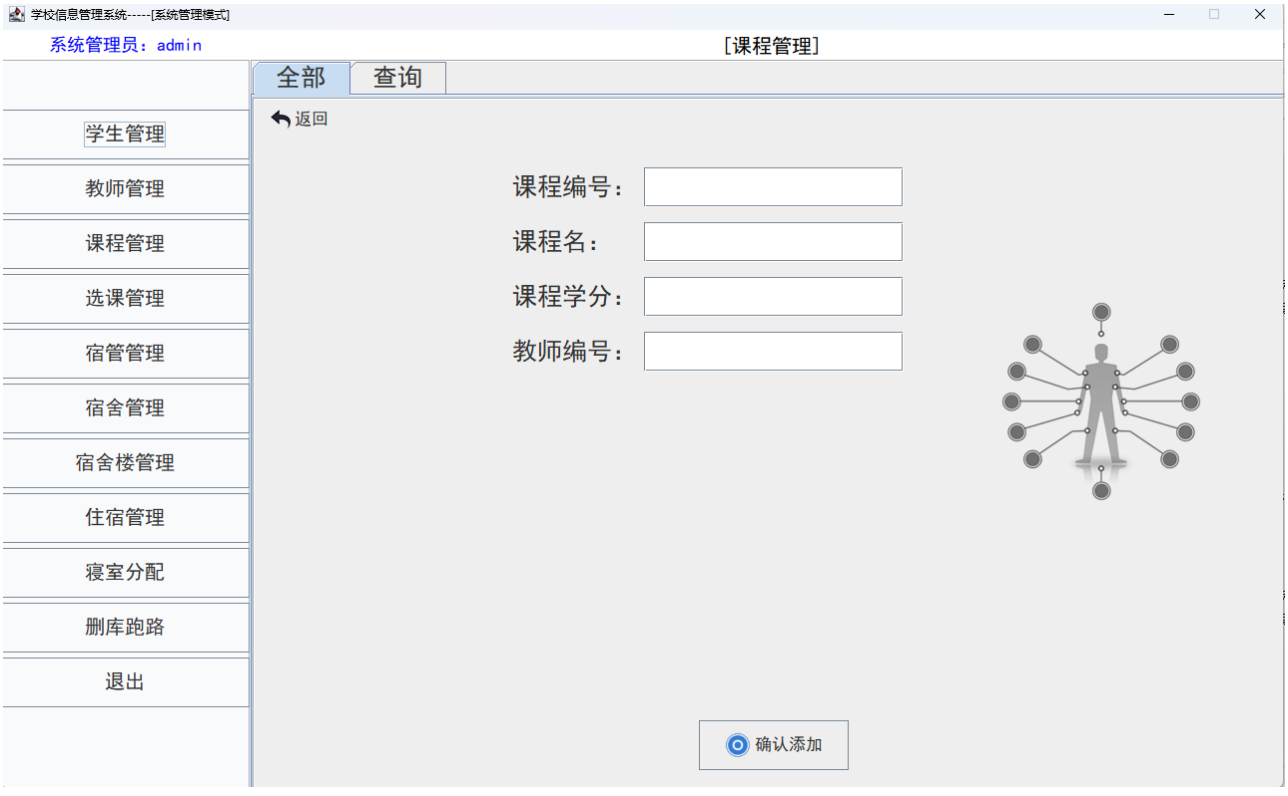
点击'导出'，即可将所有教师信息导出到excel表

4.课程管理

点击'课程管理'，即可查看所有课程信息



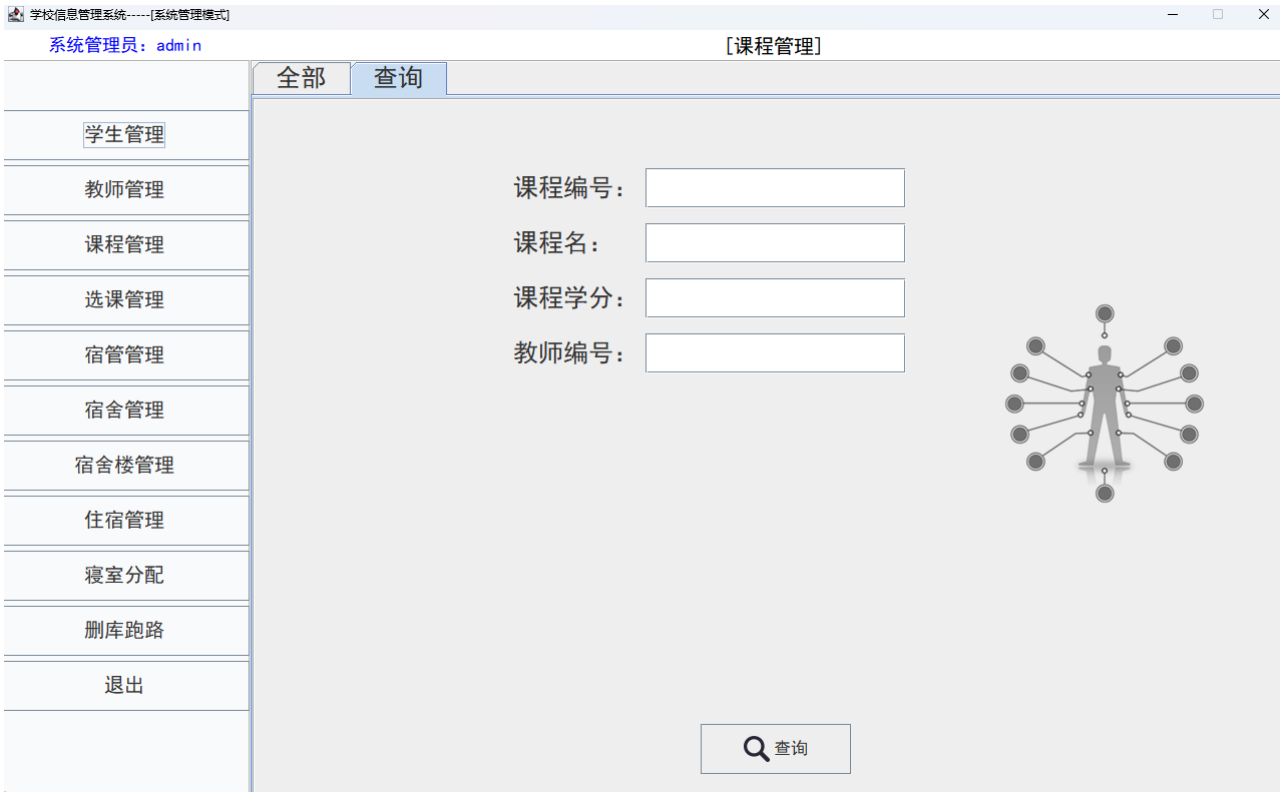
点击'添加'，即可输入课程信息进行添加一门课程



点击某门课程的'查看'按钮，可以查看该课程的信息，并且可以点击'修改课程信息'或'删除课程信息'按钮修改和删除该课程的信息



点击'查询'，可以查询指定的课程信息



点击'导出'，即可将所有课程信息导出到excel表

5.选课管理

点击'选课管理'，即可查看所有选课信息



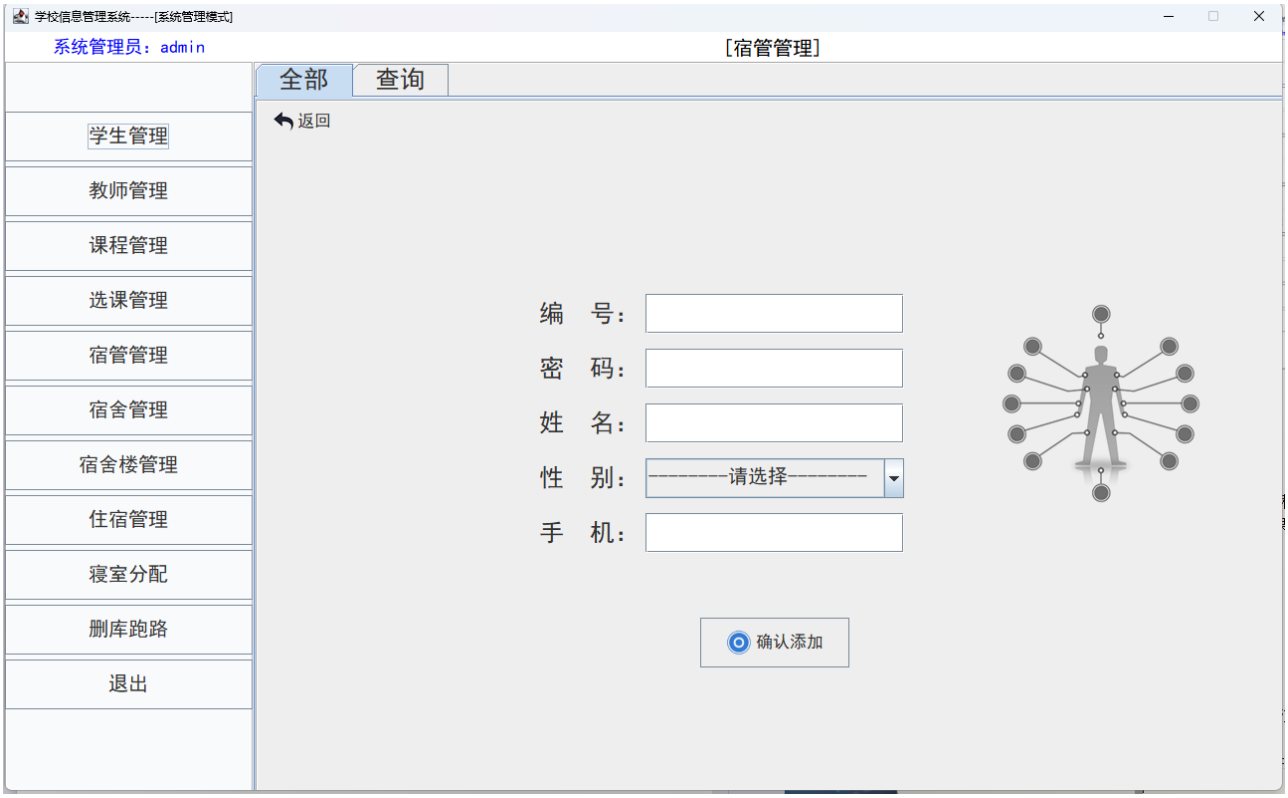
点击'导出'，即可将所有选课信息导出到excel表

6.宿管管理

点击'宿管管理'，即可查看所有宿管信息



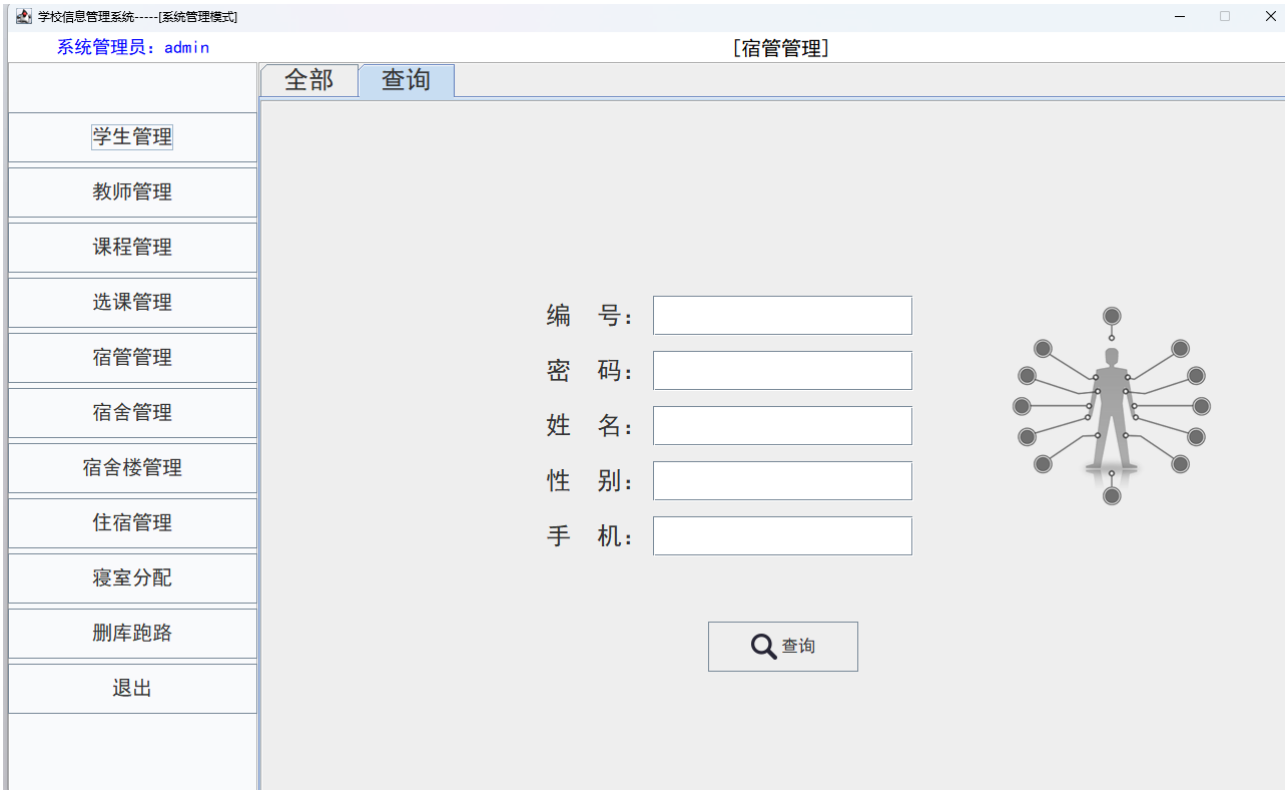
点击'添加'，即可输入宿管信息进行添加一名宿管



点击某名宿管的'查看'按钮，可以查看该宿管的信息，并且可以点击'修改宿管信息'或'删除宿管信息'按钮修改和删除该宿管的信息



点击'查询'，可以查看指定宿管的信息



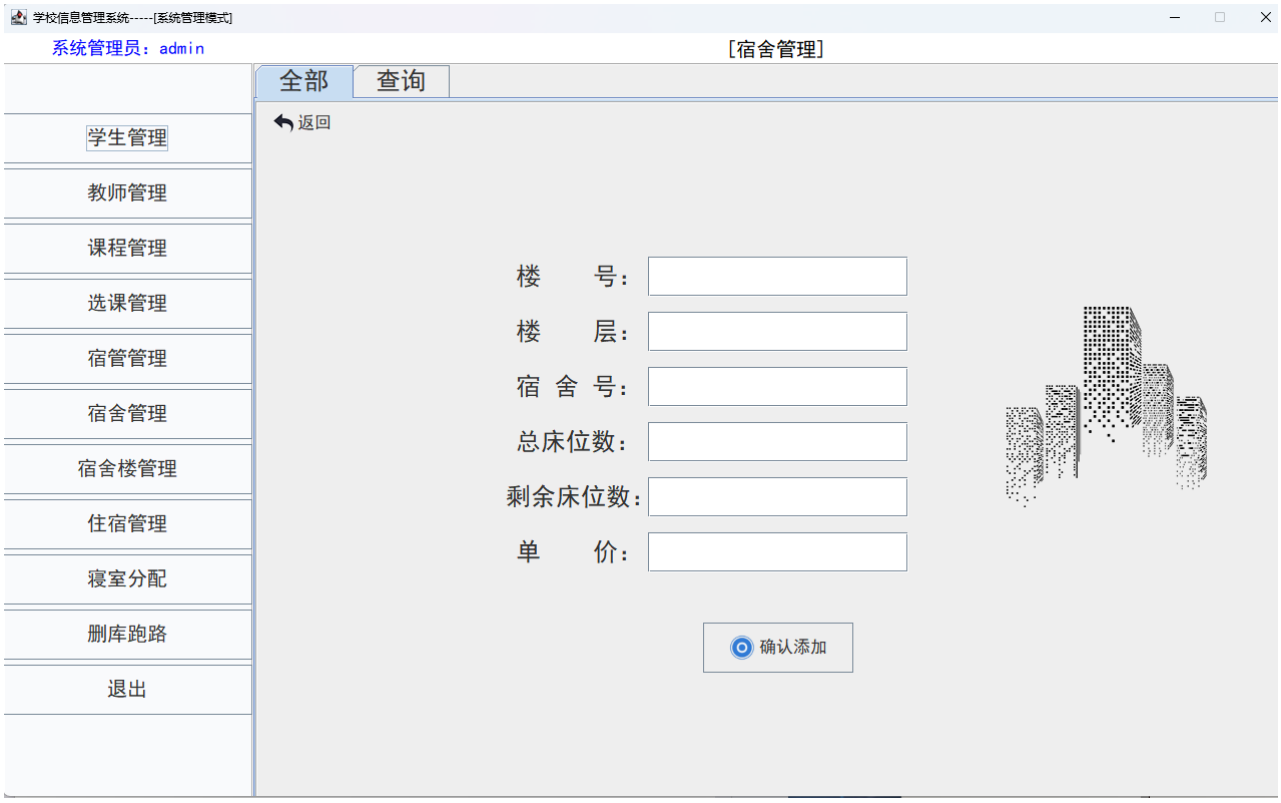
点击'导出'，即可将所有宿管信息导出到excel表

7.宿舍管理

点击'宿舍管理'，即可查看所有宿舍信息



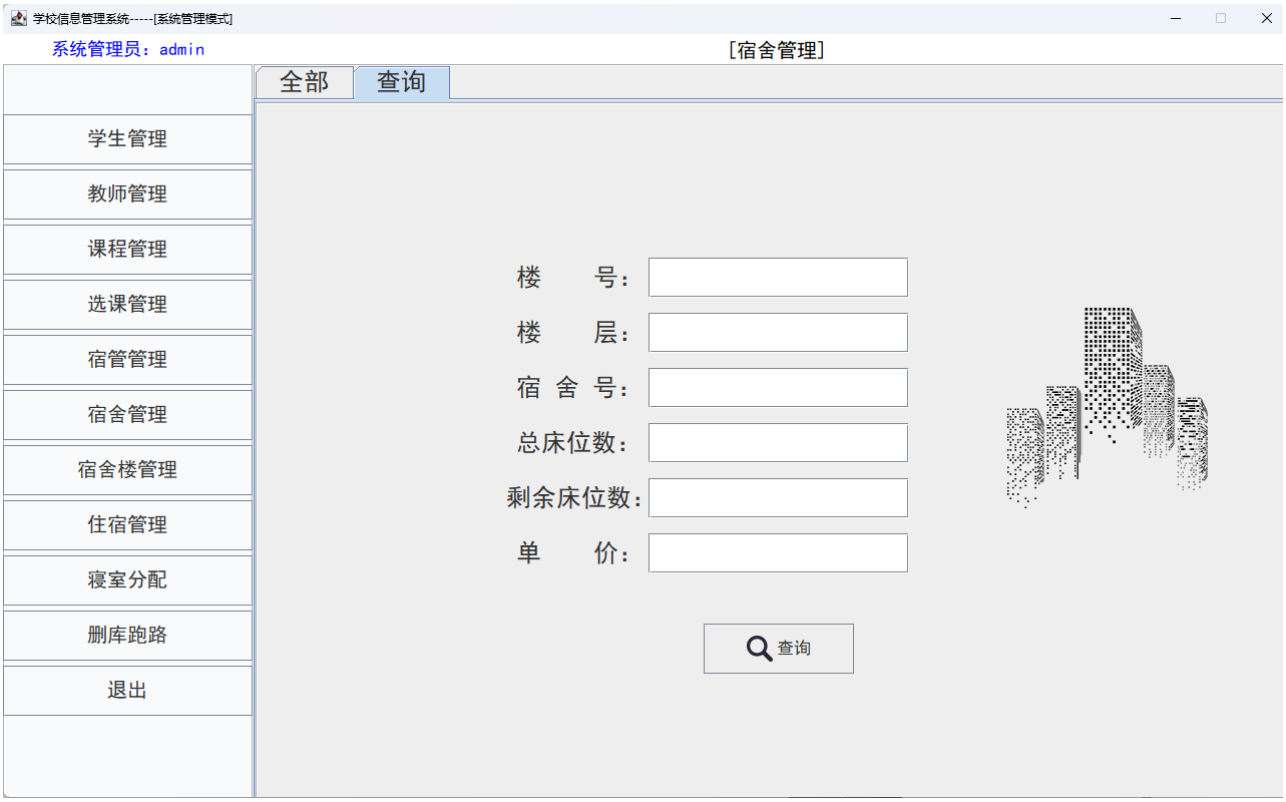
点击'添加'，即可输入宿舍信息进行添加一个宿舍



点击某个宿舍的'查看'按钮，可以查看该宿舍的信息，并且可以点击'修改宿舍信息'或'删除宿舍信息'按钮修改和删除该宿舍的信息



点击'查询'，可以查看指定宿舍的信息



点击'导出'，即可将所有教师信息导出到excel表

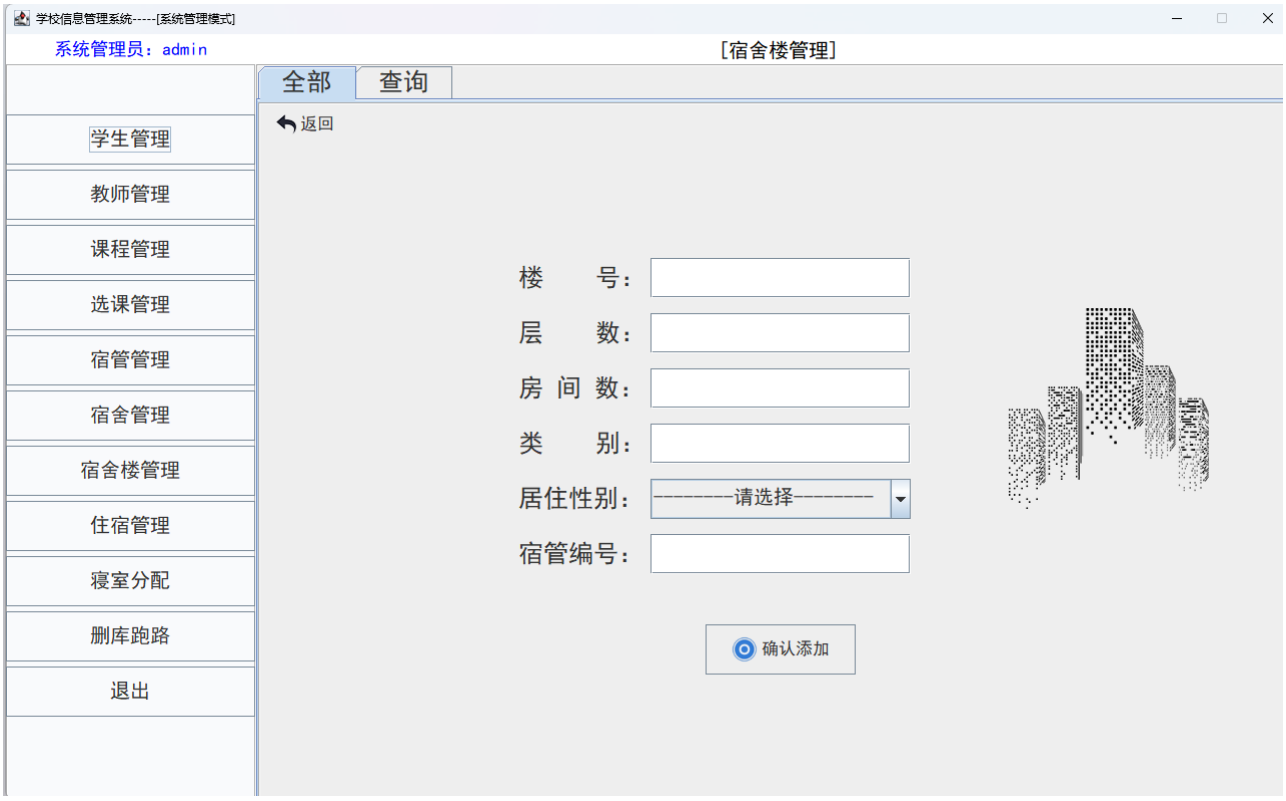
8.宿舍楼管理

点击'宿舍楼管理'，即可查看所有宿舍楼信息



楼号	层数	房间数	类别	居住性别	宿管编号	操作
TM11	8	320	普通	男	1004	查看
TM25	7	140	豪华	混合	1001	查看
TM316	7	280	普通	男	1003	查看
TM39	6	240	简朴	女	1002	查看
TM47	7	210	豪华	女	1005	查看
TM66	5	100	巨豪华	男	1006	查看

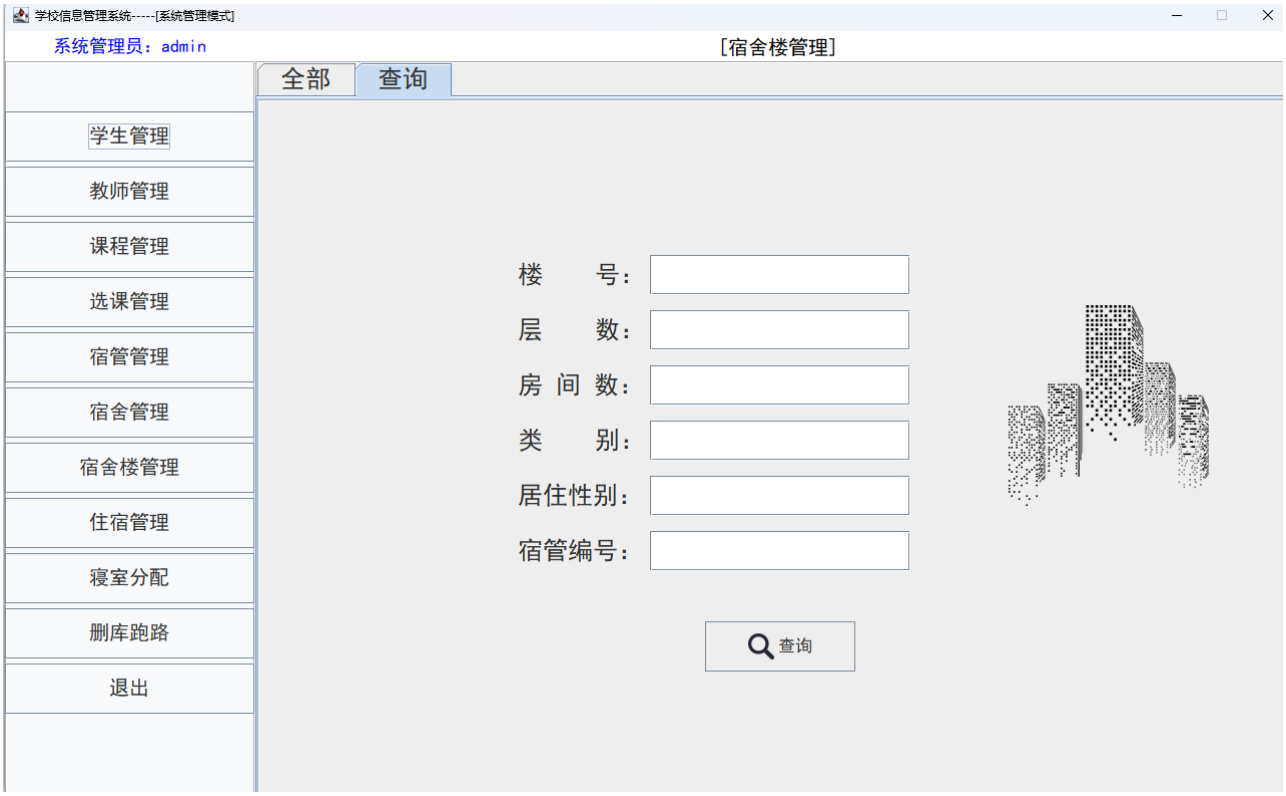
点击'添加'，即可输入宿舍楼信息进行添加一栋宿舍楼



点击某栋宿舍楼的'查看'按钮，可以查看该宿舍楼的信息，并且可以点击'修改宿舍楼信息'或'删除宿舍楼信息'按钮修改和删除该宿舍楼的信息



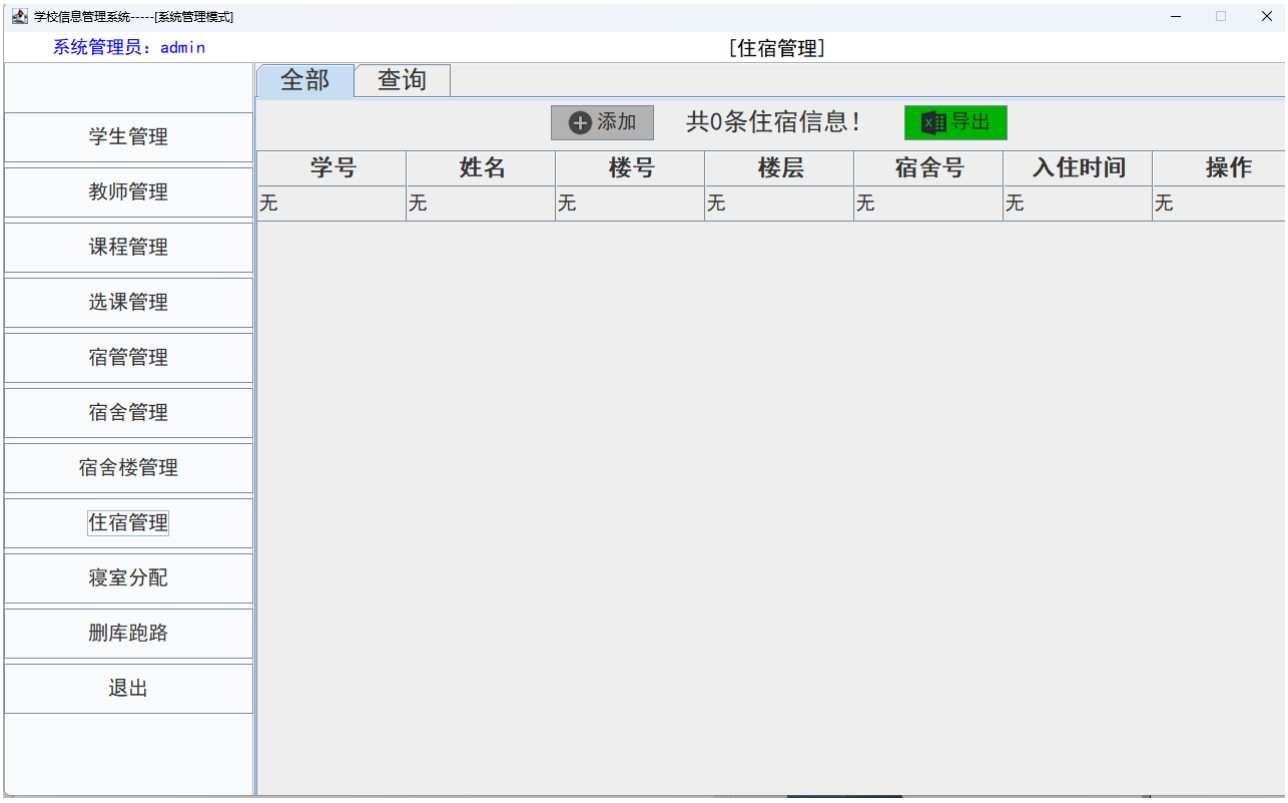
点击'查询'，可以查看指定宿舍楼的信息



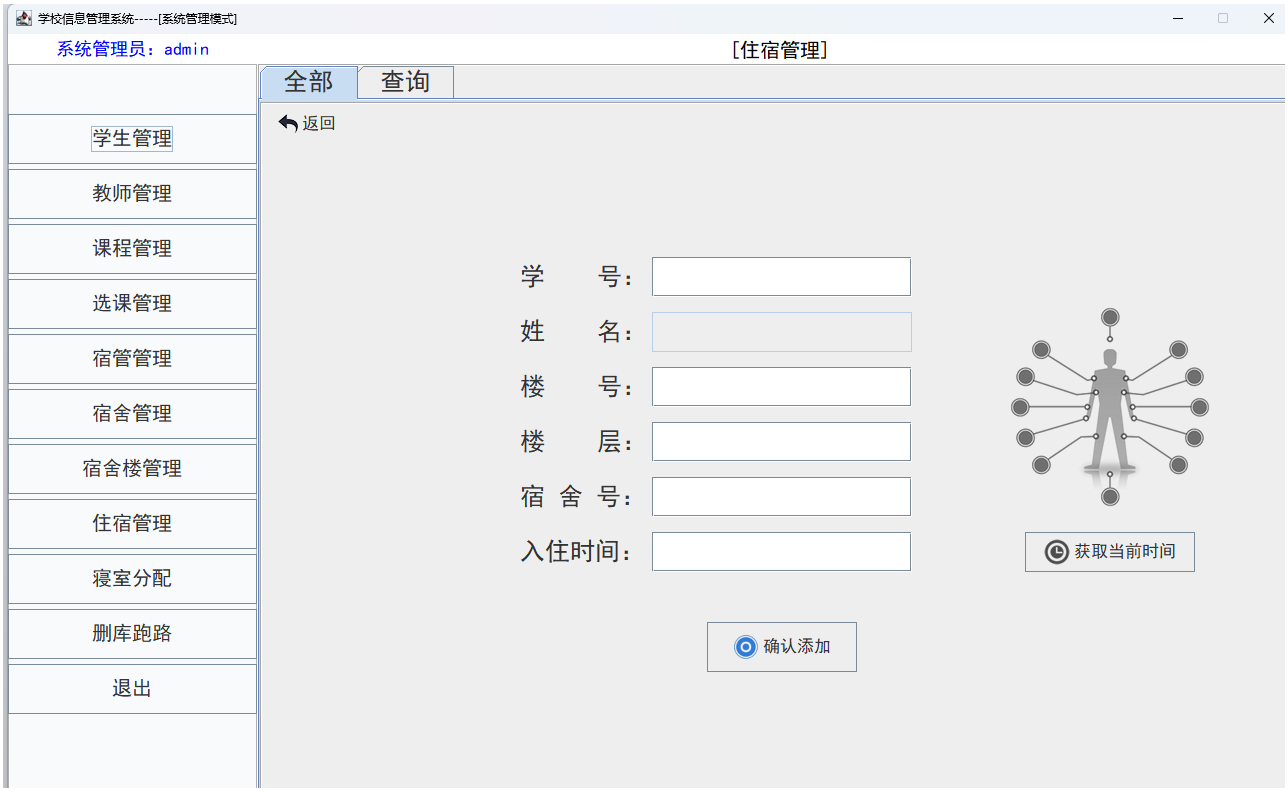
点击'导出'，即可将所有宿舍楼信息导出到excel表

9.住宿管理

点击'住宿管理'，即可查看所有住宿信息



点击'添加'，即可输入住宿信息进行添加一个住宿信息



点击'查询'，可以查询指定的住宿信息



点击'导出'，即可将所有宿舍楼信息导出到excel表

10.寝室分配

点击'寝室分配'，即可进入寝室分配界面



点击'一键自动分配'或'一键重新分配'即可对宿舍进行分配。

分配规则如：按照性别、年级、院系、班级进行分配，分配宿舍为普通宿舍
键自动分配:将为系统内未住宿的学生自动分配宿舍。
键重新分配:将删除所有住宿信息，重新为系统内的学生自动分配宿舍



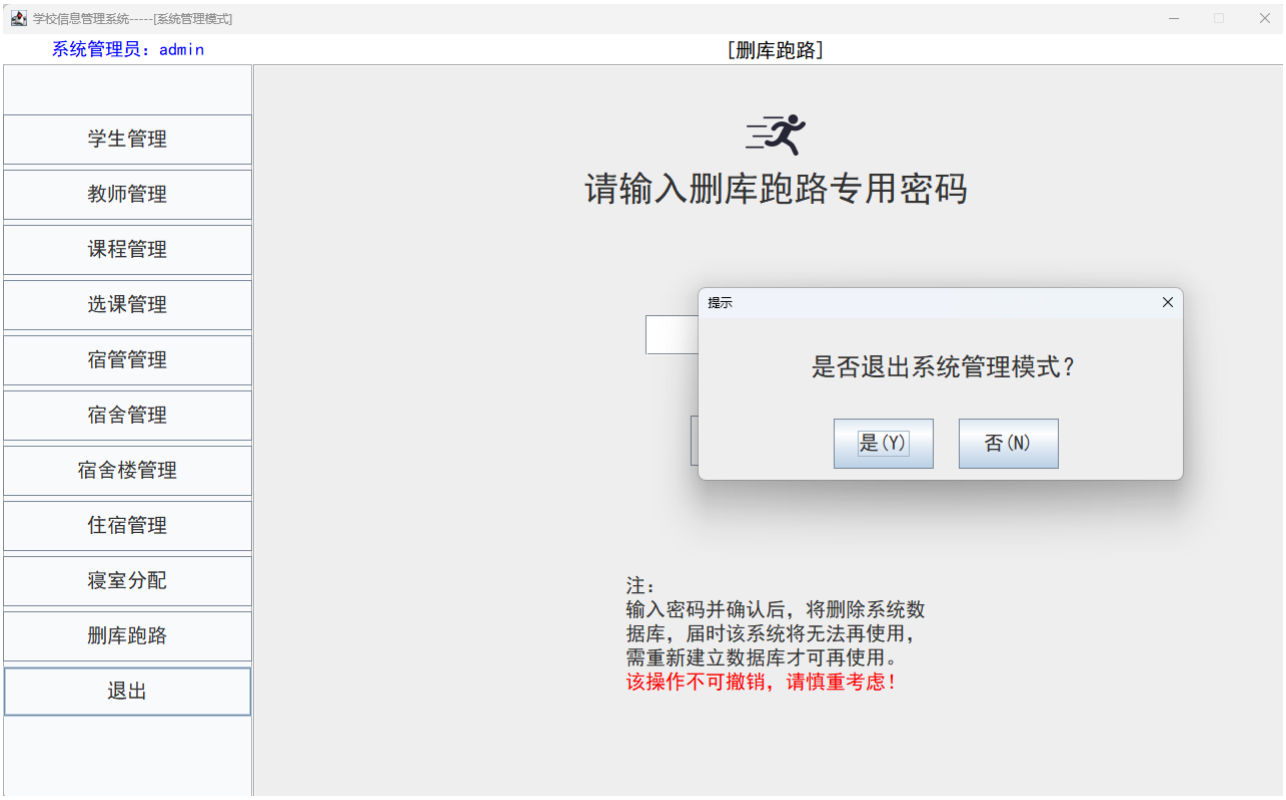
11.删库跑路

点击'删库跑路'，输入删库跑路专用密码即可删除数据库



12.退出

点击'退出'，即可退出系统管理员模式



四、总结与思考

本次大作业围绕我们在日常学习生活中常见的元素展开。我们参考教务系统，宿舍管理制度等，完成了一个学校信息管理系统，能处理四种不同身份的访问者的交互。虽然只是一个很小的信息管理系统，但是完整地经历了数据库设计开发流程，我们确实收获了许多。

我们从需求分析开始，绘制出概念结构，抽象出逻辑结构，设计出物理结构，并生成出Sql语言描述的文件，以此完成数据库设计。之后我们使用java写出可视化界面，通过Mysql语言与数据库发生交互，产生数据的流动和呈现，以此完成软件设计。最终我们实现了预期的功能。

通过此次大作业，我们更加了解数据库的重要性，使用数据库作为数据的存储管理后台，可以极大的提高数据的处理能力。我们得以在书本的基础上实际运用，在更加真实的环境下运用知识解决问题，展现“经世致用”精神。我们锻炼了实际应用能力，也锻炼了代码能力与逻辑能力。同时，在将所学转化为所用的过程中，我们对于课本知识也更加融会贯通。

在从零开始到最终设计出一个运用了数据库的应用系统，我们遇到了许多的问题和程序的报错，解决了很多bug，但是团队内的大家都很团结，最终这些问题都在我们的努力下得以解决。团队的意义和合理分工的重要性在本次开发的过程中得到了凸显，同时我们也从一个侧面看到了并行解决任务相较于串行解决任务的独特优势。

回望大作业实现的过程，虽然解决了很多拦路的难题，但是我们还有很多不足。特别是对于前后端的分离，本次实验没有实现。主要是时间受限，我们还有很多想法没有化为现实。此外有一些在企业运用上更为广泛使用的、更好的前后端结构，这些都是需要学习和可以优化的地方，在之后有时间的时候，我们团队会继续探究。

总结来看，我们解决了许多问题，学到了许多知识，但同时也意识到了还有很多不足。虽然在过程中很痛苦，但是最终看到预期的功能得以实现，还是很自豪与开心的。